基于语义的数据湖表格增强系统

本科毕业设计（论文）

摘　要

近年来，数据湖技术的蓬勃发展大幅提升了数据资源的可获得性，为自动化发现和整合语义相关的外部表格开辟了新途径。这不仅为表格增强提供了创新方法，也为提升下游机器学习任务的性能带来了新的可能。然而，现有的表格增强方法主要依赖于文本语法相似度和列名相似度进行匹配，在应对数据湖中普遍存在的语义异构性和结构异构性问题时，仍面临着巨大的挑战：一是过度依赖文本相似度进行表格发现，难以发现表格之间的隐式语义关联；二是增强过程中依赖列名或者列文本相似度进行匹配，容易造成误匹配或者大量空缺值的出现。

针对上述挑战，本毕业设计构建了一套基于语义的数据湖表格增强系统。该系统基于FastAPI后端框架、Vue3前端框架以及预训练语言模型和大语言模型(Large Language Model, LLM)技术，实现了表格预处理、相关表格发现和表格增强等全流程自动化功能。系统工作流程包括四个核心步骤：首先，运用 Embedding 模型将表格数据转化为向量表示；在此基础上，构建多层 HNSW 索引结构以实现高效的相似表格检索；随后，利用 LLM 分析筛选出的 top-k 相关表格并生成增强建议；最后，根据建议执行数据补充和空缺值填补等表格增强操作。整个流程完成后，系统输出经优化增强的表格，供下游任务使用。

系统性能评估采用 Santos 基准数据集与 WWk 数据集，从精确率、召回率及查询延迟三个维度与 D^3L、BM25 和 TF-IDF 等基线方法进行对比。实验结果显示，本系统在准确率方面相比最优基线 BM25 提升约 10%，检索速度提升约 60%。在数据增强效果方面，增强后的表格能为下游任务带来约 40%的性能提升，有力证实了本方法在相关表格发现和数据增强两个方面的优势。

关键词：数据湖，表格增强，语义驱动，预训练语言模型，大语言模型

ABSTRACT

In recent years, the rapid development of data lake technologies has greatly improved data accessibility, enabling new approaches for automatically discovering and integrating semantically related external tables. This opens up innovative opportunities for table augmentation and enhancing downstream machine learning tasks. However, current methods largely rely on textual and column name similarity, which struggle with semantic and structural heterogeneity common in data lakes, leading to issues such as failure to capture implicit semantic links and high rates of mismatches or missing values.

To address these challenges, this thesis presents a semantic-based table augmentation system for data lakes. The system is implemented using the FastAPI backend framework, the Vue3 frontend framework, and incorporates pretrained language models and large language models. It supports a fully automated pipeline encompassing table preprocessing, related table discovery, and augmentation. The system follows a four-step core workflow: First, it uses embedding models to convert table data into vector representations. Based on these vectors, a multi-layer HNSW index is constructed to enable efficient retrieval of similar tables. Next, an LLM analyzes the top-k retrieved candidate tables and generates augmentation suggestions. Finally, the system applies these suggestions to supplement data and fill in missing values. After completion, the system outputs an optimized and augmented table for use in downstream tasks.

System performance is evaluated using the Santos benchmark dataset and the WWk dataset, comparing against baseline methods such as D^3L, BM25, and TF-IDF across three metrics: precision, recall, and query latency. Experimental results show that the proposed system improves precision by approximately 10% over the best-performing baseline BM25 and increases retrieval speed by about 60%. In terms of augmentation effectiveness, the enhanced tables yield around a 40% performance improvement in downstream tasks, demonstrating the system's significant advantages in both related table discovery and data augmentation.

KEY WORDS: Data Lake, Table Enhancement, Semantics-driven, Pretrained Language Model, Large Language Model

目　　录

摘　　要.....I
ABSTRACT.....II
目　　录.....III
第一章 绪论.....1
1.1 课题背景和意义.....1
1.2 研究现状.....1
1.3 本文工作内容.....7
1.4 论文组织架构.....8
第二章 相关技术.....9
2.1 Embedding 模型.....9
2.2 Transformer 模型与对比学习.....10
2.3 BERT 模型.....10
2.4 本章小结.....11
第三章 系统详细设计与实现.....12
3.1 需求分析.....12
3.2 架构设计.....14
3.3 流程设计.....15
3.4 本章小结.....30
第四章 系统测试与结果分析.....31
4.1 测试目标与测试配置.....31
4.2 系统测试与分析.....32
4.3 本章小结.....43
第五章 总结与展望.....45
5.1 工作总结.....45
5.2 工作展望.....45
参考文献.....47

第一章 绪论

1.1 课题背景和意义

近年来，随着数据湖技术的快速发展，越来越多的组织和机构积极参与开源数据集的建设，数据资源的可获得性显著提升。然而，数据湖中的海量数据通常分散且缺乏统一管理，如何高效整合和利用这些多样化数据以支持数据科学任务仍面临巨大挑战。单一表格往往无法提供足够信息，限制了数据分析与挖掘的效果。但是，在目前表格数据的增强过程中时，用户往往面临两难选择：要么被迫在多个碎片化工具间频繁切换，构建复杂的工作流；要么耗费大量时间进行重复性手动操作。因此，构建一个基于语义的数据湖表格增强系统，自动发现并集成相关表格以提升数据科学任务的性能，具有重要的理论价值和实际应用意义。

研究人员针对相关表格发现和集成问题提出了多种解决方案。传统方法主要基于表格中词频相似度来评估表格相关性，但这种方法存在明显不足，难以准确捕捉表格之间复杂的语义关系。此外，传统方法的计算效率较低，导致相关表格检索过程耗时过长。近年来，LLM 技术取得显著进展，为表格发现和集成提供了新的解决思路。LLM 具有强大的语义理解能力，可以更深入地解析表格内容。基于这些优势，开发结合 LLM 的新型表格增强系统具有重要意义，可以实现更高效、更准确的表格发现与增强。

本论文设计并且实现了一个基于语义的数据湖表格增强系统，能快速发现相关表格并进行有效增强，从而更好地利用数据湖中的表格数据。系统通过自动化的相关表格发现与集成功能，显著减少了数据科学家在数据准备阶段的时间投入。不仅如此，系统通过增强原始表格的数据丰富性，能够提升多种下游数据科学任务的性能表现。另外，系统还简化了数据增强的操作流程，为用户提供了便捷高效的数据处理工具，通过简便友好的系统界面设计，系统有效降低了数据增强的技术门槛，使更多用户能够轻松使用表格增强功能。

1.2 研究现状

表格增强主要包含两个关键环节，分别是在相关表格发现阶段查找相关表格，和在增强过程中处理冲突问题。基于这两个表格增强过程中的核心步骤，本文的研究现状综述将围绕两个相关领域展开，分别是相关表格发现领域的研究现状和增强冲突解决领域的研究现状。

1.2.1 数据湖与相关表发现任务

数据湖是一种支持海量原始数据存储的系统架构，能够容纳多种结构类型的数据，包括结构化、半结构化和非结构化数据。数据湖不需要预先定义数据模式或结构，允许数据以原始形式直接存储。这种存储方式使得数据湖的使用十分灵活，能够适应多样化的数据类型和格式。基于这些特性，数据湖可以满足各种不同的数据分析和处理需求。

数据湖相对于传统的数据库和数据仓库具有四个主要优势。第一，它具有出色的灵活性，能适应不同数据源和多种数据格式，满足多样化的分析需求。第二，数据湖具有显著的成本效益，采用低成本的存储解决方案，特别适合海量数据的存储需求。第三，数据湖实现了数据民主化，使组织内更多成员能够访问数据资源。最后，数据湖支持高级分析功能，包括机器学习和数据挖掘等技术，为数据科学家和分析师提供了丰富的数据基础。这些特点使数据湖成为现代数据分析的重要基础设施。

而在数据湖中，相关表格是指是指与用户输入的原始表格在语义或结构上具有关联性，能够通过联合(UNOIN)或连接(JOIN)等操作进行整合的表格[2]。可联合的表格往往与原始表格在结构上相似，可以通过联合操作将多张表按行合并，扩展数据量。可连接的表格往往与原始表格在语义上相关，可以通过连接操作基于某些共同的字段，对表格进行列上的扩展，从而丰富原始表格的信息[4]。

相关表格发现任务是为了发现与查询表格有关联的表格，这个关联主要体现在语义相关性、结构兼容性和数据互补性[2]。语义相关性指相关表格与原始表格在语义层面存在密切联系或高度相似。结构兼容性表现为相关表格与原始表格具有相似的表结构，这使得它们可以通过联合或连接操作得到新的增强表格。数据互补性是指相关表格能够为原始表格补充额外的信息，可以丰富原始表格的数据特征，提升下游数据科学任务的表现效果。

1.2.2 相关表格发现

在数据湖表格增强场景中，相关表格发现问题可以被形式化定义为：给定一个源表 Q，从包含 N 张表的数据湖中检索出与其最相关的前 k 个表。最直接的实现方式是线性扫描查找，即遍历数据湖中的所有表格，并分别计算它们与源表之间的相似度得分，从中选出得分最高的 k 个表。然而，这种方法在实际应用中面临效率低下问题，例如计算开销过大和计算时间过长。针对这个问题，研究人员在相关表格发现任务中提出了多种优化方法。这些方法主要围绕两个核心目标：提升查询效率的和提升相关表格发现的准确率和召回率。根据所采用的核心方法差异，现有优化方法可以分为以下几类：

1.2.2.1 精准发现相关表格

在精准发现相关表格方面，研究人员主要关注如何更准确地量化表格间的相关性。传统方法通常依赖于表名、列名等文本层次的简单文本匹配，较难捕捉深层次的语义信息。为了捕捉深层次的语义信息，近年来出现出多种基于关系图或者是基于机器学习的语义识别技术，这些方法提升了相关表格识别的准确度，为相关表格发现奠定了坚实基础。

列语义和表格不同列之间的语义关系可以用来表示表格的语义。Khatiwad 等人[1]通过给表格每个列分配一组语义注释，并且给表格的每对列分配一组关系语义，使用关系语义表示列值之间的二元关系，从而更好地提取表格的特征，提高数据湖中表格联合搜索的准确性。同时，使用现有知识库和合成知识库来识别列之间的关系，给每个表格构建语义图，使用图中的节点表示列，边表示列之间的关系，最后通过比较列语义和关系语义的匹配分数，计算联合搜索的总体得分来确定表格之间的相似度。

通过利用数据集中的特征构建基于哈希的索引，将表格特征映射到一个统一的距离空间中，从而定义特征之间的相似度，并将其作为与表格相关性的度量，Bogatu 等人[2]提出了一种名为 D^3L(Dataset Discovery in Data Lakes)的方法来解决数据湖中相关表格的发现问题。D^3L 通过计算属性名称相似度、属性值重叠、词嵌入相似度、格式表示相似度和域分布相似度等五个特征来确定两张表格的相似度。通过这五个特征，能够综合考虑多种影响表格相似度的因素，通过有效地调整这些参数所占的比例，能够大大提升对于表格语义识别的精度，提升了相关表格发现的准确性。与 Bogatu 等人提出的方法类似，Nargesian 等人[3]提出了通过集合域、语义域和自然语言域等三种属性域来评估两个属性是否来自同一个域，从而评估两张表格的相关性。不仅如此，该论文还提出了一种综合方法，自动选择最适合每对属性的联合性评估方法，从而得到每对属性的最高相似度，以此来更加智能地捕捉表格之间地相似性。

混合整数规划(Mixed-Integer Programming, MIP)是一种数学优化方法，用于解决目标函数和约束条件同时包含连续变量和整数变量的优化问题。Chen 等人[4]提出了一个优化问题，使用混合整数规划的方法来选择最佳的相关表格集合。该论文创新性地在表格语义之外考虑了两个表格之间的兼容性，即它们是否可以通过列连接，兼容性得分由列相关性和外键约束的满足可能性组成。

哈希值不仅可以代表表格每一列的特征，还可以减少存储表格特征所需的空间。Santos 等人[7]提出了一种基于 QCR(Quadrant Count Ratio) 哈希的相关表格发现方法。QCR 哈希将

数值平面划分为四个象限，生成哈希值，使得具有相同列名且数值在同一象限的表能够匹配。最后，通过生成查询表的哈希项，检索出与查询表具有最多共同哈希项的表，从而找到相关表。Harmouch 等人[9]通过提出一种新颖的哈希方法 Xash，来编码表格所有列值的语法特征，通过编码语法特征，可以从更深层次理解表格地语义特征，更加准确地发现相关表格。

将表格文本转化为向量不仅可以更好地捕捉语义间关系，还可以综合考虑上下文语义并且可以把不同类型的数据映射到同一向量空间。Dong 等人[8]提出了 PEXESO 框架，该框架将表格的每一列都转化为一个高维向量，并基于向量之间的相似度来评估表格之间的相似度。Fan 等人[14]在此基础上又做了改进，提出了一个名为 Starmie 的框架。该框架使用对比学习的方式来训练列编码器，从而生成表格的列相关向量。在学习过程中，每个列向量不仅会考虑当前列的语义特征，还会学习当前列和其他列之间的关系。通过综合考虑列间语义关系，可以有效地减少错误匹配，提升相关表格发现的准确度。

这些方法虽然能提升表格发现的准确性，但都存在缺陷。基于列间语义关系的方法依赖知识库，其语义图构建过程复杂。D^3L 方法综合多个特征来提高精度，但特征相似度计算耗时，且需要调整参数。Nargesian 等人提出的方法能自动选择评估方式，但选择过程比较复杂。Chen 等人的混合整数规划方法需考虑表格兼容性，其求解耗时且建模复杂。Santos 等人的 QCR 哈希方法节省存储空间且检索高效，但主要针对数值型数据，对文本型数据效果较差。

1.2.2.2 快速检索相关表格

为了满足数据湖中海量表格的实时查询需求，研究者们致力于提升相关表格的检索速度。目前主流的加速方法分为索引优化和图优化这两类。在索引优化方面，常见技术包括局部敏感哈希和倒排索引查找，采用近似计算来提高发现速度。图优化则是通过构建表格相似度图结构，然后在图上进行近似查找，这种方法也能加快相关表格的发现速度。

通过哈希索引来减少搜索空间，可以极大地减少需要搜索的相关表格数量，从而加快相关表格的发现速度。Bogatu 等人[2]提出的 D^3L 方法通过在高维空间建立哈希索引的方式，去除与查询表格相似度较低的表格，只对与查询表格哈希值在高维空间中较为接近的表格进行查找，以加快发现速度。在 Nargesian 等人[3]提出的通过集合域、语义域和自然语言域等三种属性域来发现相关表格的方法中，他们使用了局部敏感哈希(Locality Sensitive Hashing, LSH)技术来加速相关表格的发现。具体而言，他们利用 LSH 对属性进行桶化，以

估计联合分布，从而高效计算联合性分数。

在数据集发现的过程中，先估计候选集合与查询集合的交集大小，再决定发现相关表格的策略，同样可以加快相关表格的发现速度。重叠集合相似性搜索以集合交集的大小作为相似性度量，与其他相似性度量方法不同，这种方法对小规模集合没有偏见，因此被应用于数据湖中相关表格的发现。Zhu 等人[5]基于重叠相似性搜索提出了 JOSIE(Joining Search using Intersection Estimation)算法。该算法适用于大规模集合和字典的场景，通过估计候选集合与查询集合的交集大小，动态决定是直接读取候选集合还是读取倒排索引列表，从而最小化数据读取成本，提升搜索效率。

HWNS(Hierarchical Weighted Network Summarization)是一张图，是一种多层图结构。图中顶点之间的连接基于它们所代表表格的相似程度。该图的第一层结构最简化，包含的顶点和边数量最少，顶点间距最大。最底层结构最密集，顶点和边数量最多，顶点间距最近。采用逐层遍历策略可以实现对查询点最近邻顶点的渐进式逼近。Fan 等人[14]将该 HNSW 索引应用于表格检索加速。他们首先计算列嵌入向量的余弦相似度来评估列间相似性，再通过加权二分图匹配算法最终确定表格间的相似程度。

然而，现有方法仍存在许多不足。D^3L 方法在高维空间构建哈希索引时容易出现哈希冲突问题。不仅如此，哈希索引的维护成本较高，且对数据分布的变化较为敏感。Nargesian 等人采用 LSH 技术时也面临问题，问题包括哈希参数选择存在困难和 LSH 哈希处理高维数据的能力有限。JOSIE 算法存在估计交集大小时存在误差、对小规模集合的处理能力有限、倒排索引列表的维护成本较高等问题。HWNS 虽然能够加速相关表格的发现，但其构建复杂度高且占用内存空间较多。

1.2.3 表格对齐与缺失修复

在得到 top-*k* 表格后，用户查询表格与相关表格进行连接或联合操作时可能列名与模式冲突问题，导致表格中存在空值，大大降低了表格增强的质量。为了解决表格中存在大量空值的问题，本节总结了现有的表格对齐与缺失值修复的方法，以提升表格增强的质量。

在自动化表格连接研究领域，研究者们重点研究减少表格空缺值数量的技术，提高连接过程的自动化水平。Khatiwada 等人[13]设计的 DIALITE 框架运用 ALITE 算法解决原始表格与 top-k 候选表格的冲突。该算法采用全析取操作设计，通过两个关键步骤实现表格连接：先进行整体模式匹配、识别并统一标记匹配列，然后执行全析取操作完成连接。这种方法既保证准确性，又可以提升自动化水平，可以有有效减少表格空缺值数量。

Chepurko 等人[10]设计的 ARDA 系统实现了端到端表格自动连接。他们采用软键机制解决键缺失问题，并运用核心集技术提升效率。但是，该系统需要依赖 Aurum 等外部系统识别可连接表格，限制了其自动化程度。相比之下，Auctus[12] 集成了数据集搜索引擎。它能自动执行连接操作并实现表格扩展，在处理数据时具有独特优势。

Dong 等人[8]设计的 PEXESO 框架采用左连接的方法扩展基表信息。该框架首先对查询表进行行采样，然后在可连接表上执行左连接操作。但该方法存在明显缺陷，因为左连接会导致连接后的表格会出现大量空缺值。针对这一问题，Zhu 等人[17]提出了改进方案。他们尝试将表格数据转换为数值类型，通过寻找统计相关的列来减少空缺值。不过，他们提出的方法在实际应用中仍存在问题。因为可能存在列不匹配的情况，最终表格仍会产生大量空值，这些空值会对后续的数据处理任务造成不利影响。

表格连接技术的发展已实现可以自动化处理的突破，但现有方法仍存在不足，通常仅支持等值连接和特定数据类型处理。当处理包含语义信息的数据时，由于缺乏语义理解能力，增强后的表格中会包含大量空值，这些空值会直接影响下游机器学习任务的训练效果。因此，开发有效的空值填充方法成为关键需求。对此，Stekhoven 等人[15]提出了一种基于随机森林的空值填充方法。该方法通过迭代的方式，利用随机森林模型对缺失值进行预测和填充。与简单的随机填充不同，这种模型能够考虑表格数据的现有特征。因此，它能对空值进行合理填充，这种处理方式可以避免破坏表格原有的特征。

对于表格空值填充问题，Yoon 等人[16]提出了一种基于生成对抗网络框架(GAIN)的方法，该方法通过生成器和判别器的对抗训练来实现数据的填充。GAIN 包括生成器(G)和判别器(D)，生成器负责填补缺失数据，判别器则用于区分真实数据和生成器填充的数据。不仅如此，GAIN 方法还创新性地引入了提示向量，提示向量能够提供缺失数据的部分信息，帮助判别器更准确地评估特定数据值的填充质量，有效解决了生成器可能无法学习真实数据分布的问题。

现有表格对齐与缺失值修复方法仍存在不足。DIALITE 框架虽然能有效整合列间关联关系，但由于采用全析取处理方式，在处理大数据集时会出现扩展性问题。ARDA 系统通过软键机制提升了连接灵活性，但需要依赖外部系统识别可连接表格，影响了系统的自动化程度。Auctus 虽然实现了端到端自动连接，但对不同类型的表格类型的兼容性较差。PEXESO 采用行采样策略加快连接速度，但仍无法彻底解决空值问题。

在缺失值填补方面，现有方法各有局限：基于随机森林的非参数方法能发现表格中的

潜在数据规律，但存在模型训练复杂、特征依赖性强的缺点；GAIN 方法利用生成对抗网络，通过提示向量增强判别器效果，但面临 GAN 模型训练不稳定、依赖高质量标签数据等问题。

1.3 本文工作内容

为提供高效便捷的表格增强解决方案，本文设计并实现了一套面向数据湖的全流程自动化表格增强系统。该系统由网页前端和服务后端两大部分构成，前端负责信息展示和用户交互，后端执行用户输入处理和表格增强操作。系统功能模块包括表格预处理、相关表格发现、表格增强和增强评估四个核心组件，共同实现全自动的表格增强流程。

本文详细阐述了系统的整体架构设计、各模块的实现细节及其采用的核心算法。在系统评估方面，通过与 BM25、TF-IDF、D^3L 等经典表格检索方法的对比实验，采用多维度指标进行定量分析，实验结果证实了本方法在相关表格发现任务上的优越性。

在实际表格测试中，系统表现出显著的性能优势：在随机表格向量生成场景下，基于 HNSW 的方法相比暴力遍历能够在仅使用 35%的检索时间内达到 90%以上的准确率；在 Santos 数据集上，基于 Embedding 模型的 Embedding-HNSW 方法实现了超过 85%的准确率；在包含 3000 张表格的测试环境中，单次表格增强操作可在 60 秒内完成，充分满足实际应用需求。

本文主要完成了以下几点工作：

(1) 构建了并实现了一套全流程的、全自动的针对数据湖表格的增强系统。该系统可以自动实现表格向量化、相关表格发现、表格增强执行与空缺值填补，打通了表格增强的整套流程。

(2) 通过系统的实验设计，将提出的 Embedding-HNSW 相关表格发现与 BM25、TF-IDF、D^3L 等经典的相关表格检索方法进行了对比分析，并采用多维度评估指标对实验结果进行定量评估，结果表明本文提出的方法在相关表格发现任务中的表现优于基线方法。

(3) 通过消融实验证明了本文使用对比学习方法训练的 Transformer 模型可以帮助更好地捕捉表格列之间的关系，增加相关表格发现的准确率，同时通过对于开源表格数据集进行实际增强测试了本系统的数据增强效果。

1.4 论文组织架构

本文章节的内容组织如下：

第一章为绪论部分，介绍了数据湖表格增强的研究现状和该课题的研究意义和内容；

第二种为系统所用相关技术，介绍了系统各个部分中运用的预训练语言模型技术和使用优势；

第三章为系统详细设计与实现，包含对系统架构、实现流程和关键算法的介绍和说明；

第四章为系统测试与结果分析，包含了对本系统在功能上与性能上的测试和对比分析；

第五章总结全文工作，并提出了一些未来可以改进的内容。

第二章 相关技术

本文设计并实现了基于语义的数据湖表格增强系统，成功实现了数据湖表格的智能增强。系统流程包含多个关键环节，为了保证表格增强质量和用户体验，系统在每个环节都面临着独特的挑战。

在表格预处理阶段，系统需将数据湖中的异构表格转换为统一结构的嵌入向量，方便高效计算表格相似度。在相关表格发现环节，系统要精准捕捉表格间的语义关系，提升发现精度。在缺失值填补阶段，系统需深入分析表格特征，保证空缺值的准确填充。

为应对这些挑战，系统同时使用了三种关键技术：词嵌入 Embedding 模型、列语义建模 Transformer 模型和缺失值填补 BERT 模型。本章将详细介绍这三种核心模型，并分析它们在各个环节的具体作用和技术优势。

2.1 Embedding 模型

2.1.1 简述

Embedding 模型是一种用于自然语言处理任务的模型，可以将文本转换为高维数值向量。Embedding 模型通过分析上下文语义关系来生成这些向量，使得含义相近的文本在向量空间中位置接近，实现了离散文本数据到连续数学表示的转化。Embedding 模型的应用场景也十分广泛，支持语义搜索和文本分类等任务。与传统方法相比，Embedding 模型具有显著优势，它能够有效处理一词多义现象，并能准确捕捉词语间的语义关联关系[18][21]。

2.1.2 模型优势

在查找相似文本时，相对于直接处理文本，Embedding 模型具有以下优势：

(1) 更强的文本泛化能力：处理文本向量可以解决一词多义和同义词问题，通过向量之间的相似度而非文本来评估语义相关性，可以更好地捕捉表格之间的相似性。

(2) 更少的空间占用：Embedding 模型将稀疏的离散文本压缩为高维稠密向量，保留核心语义特征，减小存储空间占用。

(3) 更高效的相似文本检索：向量相似度计算的时间复杂度远低于文本直接匹配，适合海量数据场景，且支持分布式计算与索引优化。

在相关表格发现任务中，传统的文本匹配方法存在明显不足，无法有效解决一词多义和同义词问题。Embedding 模型提供了更好的解决方案，它可以将文本匹配转化为向量相似度计算，这更准确地识别表格间的语义关联。不仅如此，该技术还具有占用存储空间小、

计算速度快的优势。这一技术使得海量数据湖表格的快速检索成为可能。例如，HNSW[14]就使用表格向量来计算表格间相似度，通过合理的索引设计和近邻计算优化，提升了大规模表格近邻的检索效率，使得系统最终能够高效地找出与查询表格最相关的表格集合。

2.2 Transformer 模型与对比学习

2.2.1 简述

Transformer 模型是一种基于自注意力机制的神经网络模型架构，采用多层堆叠的编码器和解码器模块进行建模。Transformer 模型包括四个核心组件：多头自注意力机制、前馈神经网络、残差连接和位置编码，这些组件共同构成了 Transformer 的基础结构[19]。

对比学习是一种自监督学习方法，通过比较样本之间的相似性来学习有效的特征表示。其核心思想是将相似样本拉近，不相似样本推远，常用的方法包括 SimCLR[20]等。对比学习的方法被广泛应用于图像、文本和多模态领域，是一种效果出色的无监督学习方式。

2.2.2 模型优势

Transformer 模型采用多头注意力机制来捕捉输入文本间的语义关联关系。在表格语义相关任务中，这种多头注意力机制具有独特优势，它能够自动识别并捕捉不同列之间的语义关联。因为 Transformer 可以自主建模上下文信息，所以这在应用 Transformer 模型时不需要人工预先定义列间关系。这种自注意力机制使得 Transformer 模型适用于表格相似度计算这种无监督任务学习。

对比学习是一种高效的无监督学习方法。通过增强数据构建正负样本对，避免了大量人工标注的需求。不仅如此，使用对比学习的方法来训练 Transformer 模型使得模型更加擅长捕捉细粒度的语义差异。在表格数据处理场景中，该方法进一步优化了模型的列语义特征的提取能力。通过上述改进，能有效提升模型学习语义特征的能力，最终使得系统在相关表格发现任务中的准确率获得显著提升。

综上所述，Transformer 模型为系统提供了强大的列语义自动建模能力，而对比学习则显著提升了模型区分不同语义的能力，两者结合为表格间深层语义关系挖掘与相关表格发现提供了有力支撑。

2.3 BERT 模型

2.3.1 简述

BERT(Bidirectional Encoder Representations from Transformers)模型是一种由 Google 公

司提出的预训练语言模型。BERT 模型基于 Transformer 架构，采用双向编码器结构来建模上下文信息。BERT 模型一般被用于自然语言处理任务，包括问答系统、自然语言推理和文本分类等下游应用[18]。

2.3.2 模型优势

在表格空缺值填写任务上，相比于其他文本空缺值填充模型，BERT 模型具有以下优势：

(1) 上下文理解能力强：BERT 通过掩码语言模型(Masked Language Model, MLM)进行训练，该训练方式使得模型可以在预测空缺词语时同时利用其左右两侧的上下文信息。与传统的从左到右或从右到左的单向语言模型不同，BERT 模型能够更全面地捕捉上下文语义关联，因此在处理缺失值时更加准确[18]。

(2) 模型结构适配：BERT 模型的 MLM 训练本质上是随机屏蔽输入中的部分 token 并预测其原始内容，这与表格数据缺失值任务的处理高度一致。因此，可以自然地将缺失值视作掩码输入，BERT 可直接应用于此任务而无需修改模型结构[18]。

综上所述，BERT 模型在表格空缺值填补任务中具有明显优势。该模型具有强大的上下文建模能力和掩码语言建模机制，其双向编码结构能够同时利用缺失值前后的上下文信息，显著提高了缺失内容推断的准确性。不仅如此，BERT 模型训练时的掩码机制与缺失值填充任务天然匹配，这使得模型无需特殊改造就能直接应用于此类任务。相比传统填补方法，BERT 的填补结果的准确率高，语义一致性结果更好。这些特点使 BERT 成为表格空缺值补全的高效解决方案。

2.4 本章小结

本章详细介绍了基于语义的数据湖表格增强系统种使用的核心技术。首先时 Embedding 模型，该模型在表格相似度计算中发挥关键作用，显著提升了语义匹配的泛化能力和计算效率，为大规模表格检索提供了基础支持。然后介绍了 Transformer 和对比学习的结合应用，这种组合增强了模型对列间语义关系的建模能力，实现了更细粒度的表格相似度计算。最后介绍了 BERT 模型在空缺值填补中的优势，该模型利用强大的上下文建模能力和掩码机制，大幅提高了填补任务的准确性。这三种技术协同工作，共同构建了基于语义的数据湖表格增强系统的模型基础。

第三章 系统详细设计与实现

基于语义的数据湖表格增强系统旨在高效发现数据湖中的表格，并对用户查询的表格进行智能增强。系统通过自动化流程实现多个功能，包括表格输入、语义建模、相关表格发现、增强建议生成以及空缺值填补。这五个功能共同提升了表格增强的质量与增强后的表格质量，为下游分析任务提供支持。

系统包含五个关键处理步骤：数据预处理、存储管理、相似表格检索、表格增强执行和空缺值填补。本章首先分析系统的功能需求，具体涵盖五个方面：表格预处理、数据存储、相关表格发现、增强执行和空缺值填补。然后基于前一章的技术框架，详细说明系统架构设计和系统的运行环境和整体处理流程。最后，详细论述组成系统的表格预处理模块、相关表格发现模块、增强执行模块和空缺值填充模块的功能设计和实现，涉及模块的内部逻辑和算法设计。

3.1 需求分析

3.1.1 表格预处理

功能需求：表格预处理实现对数据湖中表格的高效表征与相似度建模。在建模过程种，系统首先利用预训练的 Embedding 模型对数据湖中所有表格进行向量化编码，将每一张表格转换为特定长度的向量。然后，基于表格间的向量相似度，获得表格间相似度。最后，系统使用此相似度，进一步构建 HNSW 近邻图，以实现高效的相似表格检索与相关表格的快速发现。

性能需求：表格预处理模块应具备良好的稳定性和可用性，能够在处理不同规模表格时保持高效性能。在向量化阶段，要求该模块能够在合理时间内准确提取表格的语义特征，并将表格每一列编码为固定长度的向量。最后，本模块在应在可接受的时间和空间复杂度内完成相似度计算和 HNSW 图构建。

3.1.2 相关表格发现

功能需求：本模块在系统中负责基于用户查询表格进行相关表格查找，需要从数据湖中高效发现与用户查询表格相似度高的候选表格。模块应根据用户提供的查询表格及查询参数(返回数量 k 等)，从构建好的 HNSW 图结构中检索出 top-k 相关的表格，并按相似度排序返回，以供下游的数据增强模块进一步使用。

性能需求：因为数据湖中表格数量很多，为保证系统的响应效率与用户体验，本模块

需在保持较高检索准确率的同时，尽可能缩短查询时间。由于 HNSW 图结构在构建与搜索过程中引入了近似搜索策略及一定的随机性，因此本模块对查询结果的一致性与持久性不作强制要求。但本模块仍需要具有良好的稳定性和查询性能，以支持实际应用场景中对效率和准确率的平衡需求。

3.1.3 增强执行

功能需求：在识别好相关表格后，增强执行模块需要根据已发现的相关表格与用户查询表格执行表格增强操作。模块应能够根据提取的增强参数以及由 LLM 提供的增强建议，对原始表格进行准确且合理的增强处理，并输出增强后的表格。

性能需求：该模块需要确保高可靠性。由于 LLM 的输出存在不确定性，模块必须遵循两个关键要求：首先，要严格按照预定义的格式规范处理输出；其次，要合理利用 LLM 提供的增强建议，确保最终结果符合要求。此外，还需具备完善的异常处理机制，能够正确应对和处理增强过程中可能出现的各种问题，保障结果正确性和整个系统的稳定运行。

3.1.4 表格空缺值填充

功能需求：在增强操作完成后，表格中可能存在空缺值。本模块需正确识别表格中各列的数据类型，并采用适当的方法对空缺值进行填充。同时，填充过程中应尽可能保留原始表格的语义特征与结构特征，避免修改表格语义，影响下游机器学习任务的训练效果。

性能需求：表格空缺值填充模块需要满足处理高效性和可靠性。在功能方面，模块需要准确识别表格列的数据类型，并根据识别结果使用相应的填充策略，完成缺失值的合理补全；在模型调用方面，由于模块处理文本空缺值时需要调用预训练的 BERT 模型，因此必须确保模型调用的稳定性和结果准确性。不仅如此，模块还需要在保证填充质量的基础上，将填充速度控制在可接受的范围内。

3.1.5 数据存储

功能需求：本模块在整个系统中负责管理数据，需要统一存储与维护所有数据，包括数据湖中的原始表格数据、表格向量化后的嵌入表示，以及用户相关的全部数据。模块需提供对外统一的数据访问接口，支持对上述各类数据的插入、更新、删除与查询操作，并对这些操作进行封装，确保系统其他模块能够高效、稳定地访问和操作所需数据。

性能需求：本模块应保障数据管理过程中的高可靠性，包括对数据完整性、一致性、持久性与容错性的严格保证。在面对系统异常等情况时，能够确保数据不会丢失或损坏，提供持续稳定的数据服务能力。

3.2 架构设计

系统采用分层架构设计，包含四层结构：交互界面层、业务处理层、数据存储层和物理资源层。原型系统设计如图 3-1 所示。交互界面层处理用户与系统的交互，包含三个主要页面：登录注册页面、表格增强页面和下游任务评估页面，这些页面引导用户完成表格增强流程和效果评估。业务处理层是系统核心，包含相关表格发现和表格增强两大模块。在相关表格发现过程中，系统使用预训练的 Jina Embedding 模型和列相关 Transformer 模型将用户上传的表格转换为 Embedding，并通过预处理阶段构建的 HNSW 图在线检索 top-k 表格以支持增强任务。在表格增强阶段，系统远程调用 LLM，根据相关信息自动完成表格增强和空缺值填补。数据存储层负责存储数据湖表格和用户数据，预处理阶段会存储转换后的表格向量数据，这可以显著缩短用户在线查询时间，提升了用户体验。物理资源层包括处理器、磁盘等硬件，为系统运行提供基础支持。

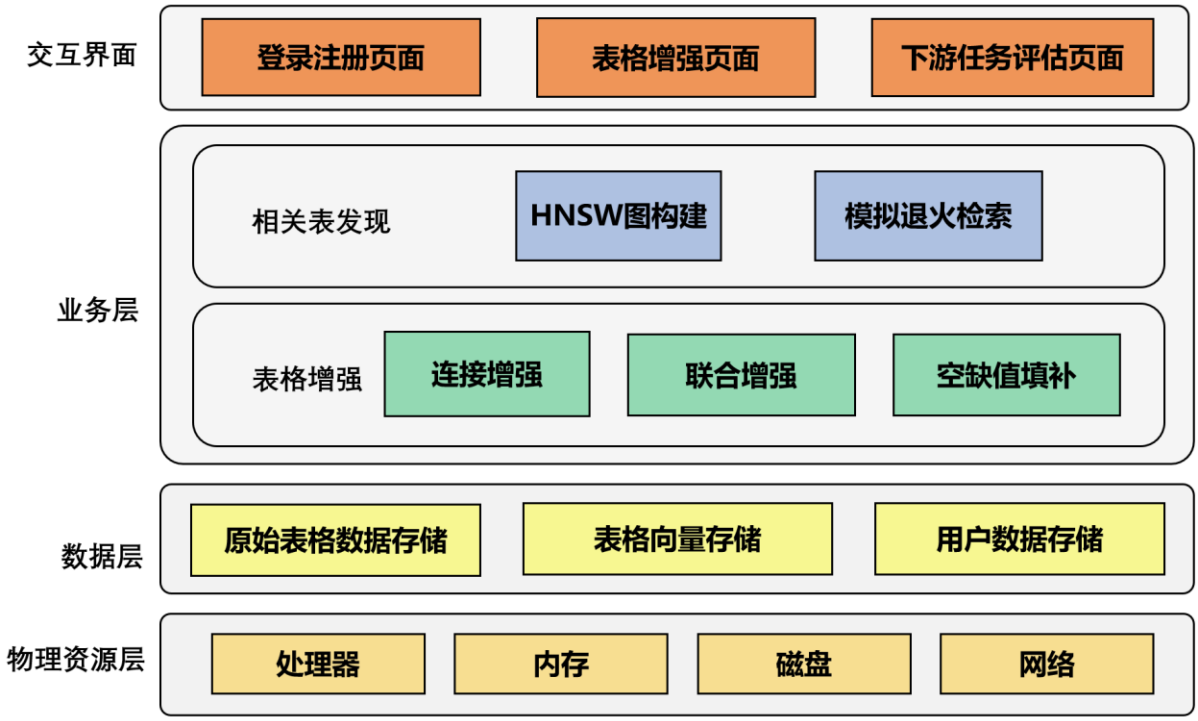


图 3-1 系统架构设计

系统在实际部署中使用前后端分离架构，前端基于 Vue3 框架实现，负责页面展示与用户交互；后端基于 FastAPI 框架实现，负责处理业务逻辑、存储数据与提供接口。系统所需的所有预训练模型全部部署在本地后端，并且使用 PyTorch、Pandans 等主流库进行代码编写。

在后端系统中，由于增强表格时经过数据预处理、相关表格发现、增强执行与空缺值

填充四个步骤，所以可以按照处理表格的各个步骤构建自动化系统。系统后端架构图如图 3-2 所示，后端系统分为预处理部分和用户即席查询部分两个部分。

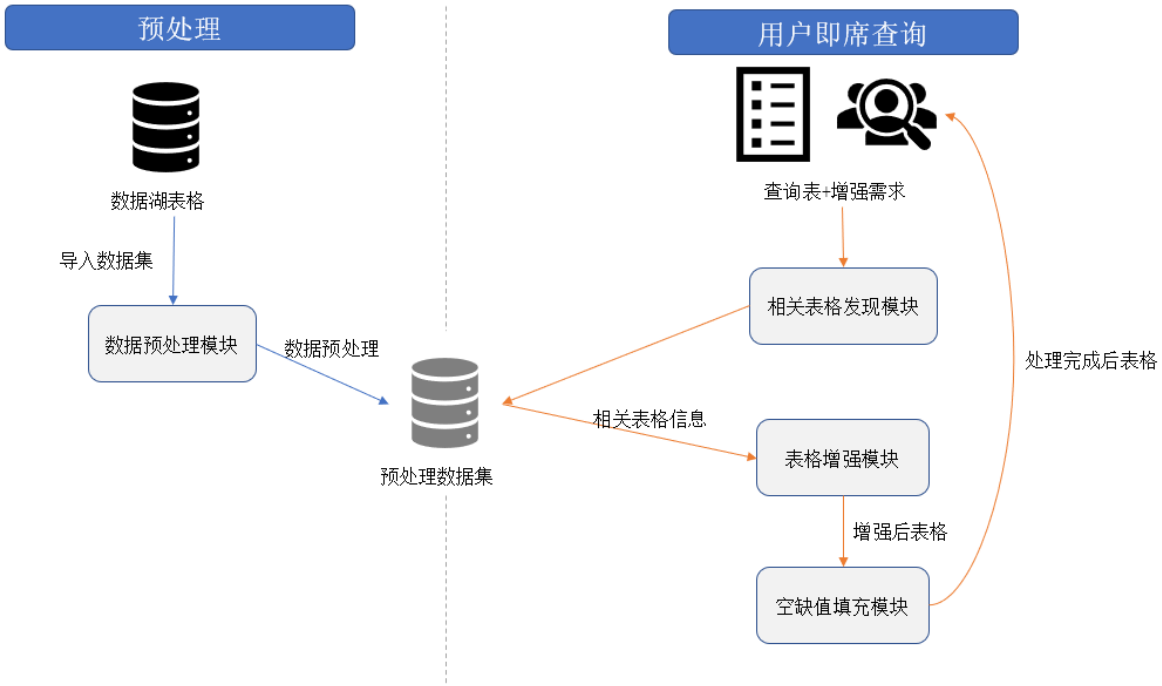


图 3-2 后端系统架构图

后端系统主要由三个功能模块构成：数据预处理模块、相关表格发现模块、表格增强模块。为了实现表格增强流程的全流程自动化处理，系统在初始化阶段将自动对数据湖中的所有表格进行向量化处理，并构建基于 HNSW 算法的索引图。

在用户即席查询阶段，后端系统通过预设工作流实现各模块的高效协同。首先，由相关表格发现模块利用构建好的 HNSW 图检索与用户查询表格相关的表格信息，并将其传递至表格增强模块；随后，表格增强模块首先结合增强需求、相关表格信息以及由 LLM 提供的增强建议，对用户表格执行增强操作，然后对增强后表格中的空缺项进行补全处理，完成后的表格将作为最终结果返回给用户。

3.3 流程设计

3.3.1 总体流程

图 3-3 所示的系统软件流程图展示了后端系统架构设计和具体实现组件。后端系统主要包含了数据库 MySQL、表格向量化 Embedding 模型、表格增强组件 Pandas 和 LLM 以及空缺值填充 BERT 模型。

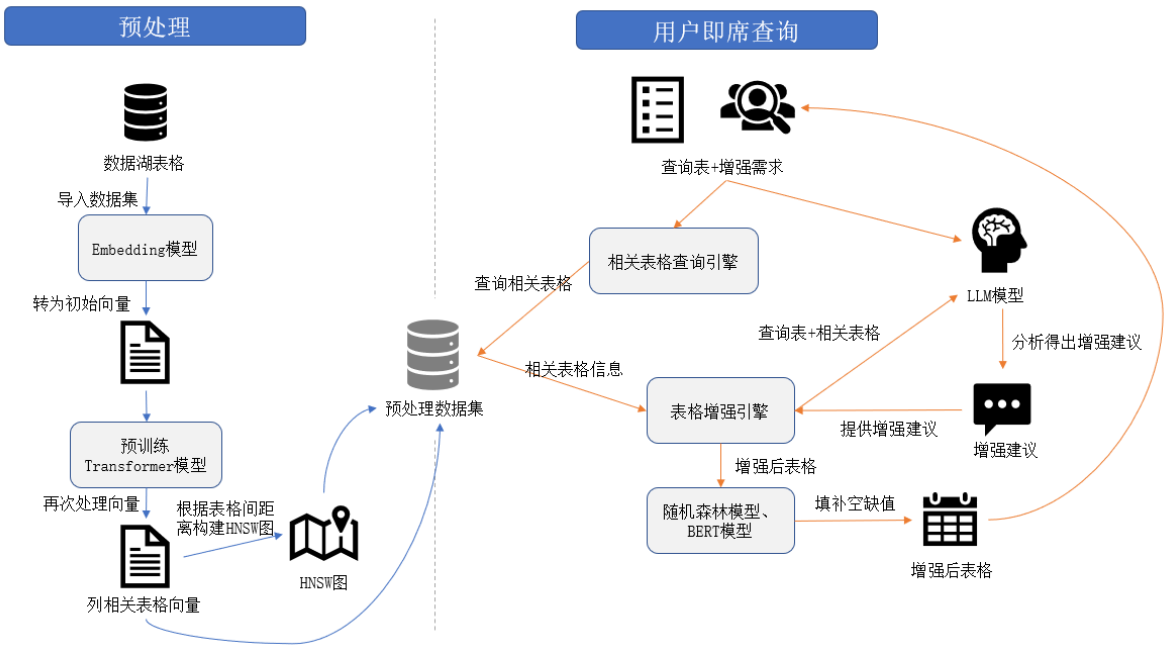


图 3-3 后端系统软件流程图

3.3.2 表格预处理

3.3.2.1 简述

表格预处理模块需要在整个系统运行的最初阶段运行，将表格数据通过两次转化转换为向量数组。因为文本向量化操作需要使用 Embedding 模型，表格列间语义关系关联需要使用 Transformer 模型，所以本模块以这两个模型为核心进行设计和实现。

Jina Embeddings 是由 Jina AI 公司提出的文本向量化模型，主要用于将文本转换为高维向量。Jina Embeddings 模型基于 Transformer 架构，并且在训练中结合了对比学习与多任务学习技术，使得其在处理语义文本匹配、向量检索等任务中具有良好的性能。该模型可以接受至多长度为 8192 长度的 token 输入，同时具备良好的推理速度和可部署性，适用于生产级应用[21]。

由于到表格间的语义关联具有较强的隐含性和表格内列间依赖性，难以通过比较显式的方式进行建模，因此在训练用于建模列间关系的 Transformer 模型时，传统的监督学习方法难以直接应用。因此，系统采用了对比学习的方式，通过对比学习构造正负样本对训练的方式，使模型能够在无显式标签的条件下自动学习列向量间的语义关系。

在训练过程中，使用归一化温标交叉熵损失函数(NT-Xent Loss)作为损失函数。该损失函数可以增强模型对正负样本对的区分能力，同时通过 Transformer 的多头自注意力机制

[19]，能够有效捕捉列向量之间的深层语义关联，从而提升表格语义表示的准确度，进一步增强相关表格发现模块的准确性。

在获得代表表格的向量数组后，系统首先使用自定义的表格距离公式，计算不同表格之间的整体相似度。基于这些相似度结果，系统构建 HNSW 图结构来建立表格之间的相似度。当用户提交查询表格时，系统可以基于 HNSW 图并且采取类似模拟退火的策略来发现相关表格，为后续增强操作提供高质量的相关表格集合。

3.3.2.2 表格列编码

本系统在将表格列转换为向量时，最初采用了一种简单方法，就是将某一列中所有行的内容通过空格拼接成连续文本，作为 Embedding 模型的输入。但这种方法存在明显缺点，典型的预训练 Embedding 模型通常都有输入长度限制，比如常见模型最多只支持 512 个 token。虽然 Jina Embedding v2 base-en 模型支持更长的输入(8192 token)[21]，但在实际应用中仍会遇到问题，某些表格列的文本长度可能超过这个上限，甚至达到上万 token。

为了在满足模型输入长度限制的同时尽可能保留表格列的语义特征，系统设计了一种基于 TF-IDF 的列内容截断方法。该方法首先遍历全部表格，计算全局词汇的逆文档频率(IDF)，随后对于表格列的每一行，计算其 TF-IDF 得分总和，作为该行的得分。最终，按照行得分从高到低进行排序，并依次拼接直至达到模型可接受的最大 token 长度，忽略剩余内容。

该方法在保证了得到的 Embedding 的质量的前提下，有效控制了输入长度，保留了更加重要的关键内容，从而为后续的列向量表示与表格相似度建模奠定了基础。该处理流程的详细步骤如算法 1 所示。

算法 *1*：获取表格的文本表示

Input: 表格 T, 单词得分字典 IDF, 最长 token 输入长度 m
Output: 表格文本 column_texts

foreach row r ∈ T **do**
    $\text{row_score}_r = \sum_{\text{token t} \in r} \text{IDF}_t$
column_texts = []
max_lenth = max(len(column_texts_c)) **for** column c∈T
while max_lenth < m **do**
    row r = (row has max row_score)
    foreach cell c∈r **do**
        $\text{column_texts}[\text{name}_c]$.appned(c)
    $\text{row_score}_r = 0$
    update max_lenth
return column_texts

为了验证 TF-IDF 算法在保留表格语义特征上的有效性，本文在三个数据集上进行了比较。实验中，对比了采用 TF-IDF 算法选取行与随机选取行所得到 Embedding 在相关表格发现任务中的准确性。实验结果如表 3-1 所示。

表 3-1 表格截取算法准确率对比

数据集	候选表数量	TF-IDF	RANDOM
Santos small	1	**1.000**	0.980
	5	**0.812**	0.801
	10	**0.732**	0.724
WWk small	1	0.035	**0.036**
	5	**0.295**	0.245
	10	**0.427**	0.389
WWk large	1	**0.036**	0.035
	5	**0.245**	0.200
	10	**0.386**	0.369

由表 3-1 可见，在三个数据集上，TF-IDF 算法的表现均优于随机选取行的方式，表明该算法在减少输入内容长度的同时，能够更有效地保留表格的语义特征。

最后，将得到的每一列文本输入到 Embedding 模型中，即可对于每一张表格得到其对应的向量数组。

3.3.2.3 对比学习

为更有效地捕捉表格列之间的语义特征，并兼顾其语义特征的隐含性与关联性，本文采用 Transformer 模型结合对比学习的方法对表格特征进行建模。训练目标在于拉近语义相同或相似表格之间的距离，同时拉远不相似表格之间的距离。

为了实现对比学习的目标，定义操作 *E(s, n)*代表对于一张表格进行增强，用于对表格进行增强处理。对于所有的数据湖中的表格$t \in T$，均生成一个增强后的副本。执行后增强操作后，所有表格都将对应一个原始表格t_{ori}及其增强后的副本t_{aug}。随后，依据上一节所述的方法，将它们分别转为向量表示Z_{ori}和Z_{aug}。增强操作的步骤如算法 2 所示，表格增强前后对比如图 3-4 所示。

算法 *2*：增强单张表格

Input: 表格 T, 交换单元格数量 s, 噪声大小 n
Output: 增强后表格 T

```
for i from 1 to s do
    c = column random chose from T
    c1,c2 = two cells random chose from c
    swap c1,c2
foreach column c∈T do
    if type_c = number do
        inject noise into column c according to noise level n
return T
```

country	capital	population
China	Beijing	1,412,000,000
Canada	Ottawa	40,000,000
Japan	Tokyo	125,000,000



country	capital	population
China	Beijing	1,412,000,123
Japan	Ottawa	40,000,456
Canada	Tokyo	125,000,789

图 3-4 表格增强前后对比图

接下来，对比学习增强过程将利用对比损失函数训练 Transformer 模型，训练过程如算法 3 所示。

算法 *3*：训练列相关 *TRANSFORMER* 模型

Input: 数据湖表格向量 Z, 增强后数据湖表格向量 Z', 学习率 θ, 每批样本数 b, 训练轮数 e
Output: 训练后模型 model

```
model = Transformer model with random parameter
for ep from 1 to e do
    random split Z,Z' into batches {B_1, ..., B_|D|/b}
    for B,B' ∈ {B_1, ..., B_|Z|/b} do
        loss = L_contrast(Z_B, Z_B')
        model = backward(model,θ,∂loss/∂model)
return model
```

本文对于损失函数的定义如下。损失函数使用交叉熵损失函数，对于在同一个批次中的原表格数组 B 和增强后表格 B'，对于来自同一种表格的向量z_i和z'_i，单对损失$l(z_i, z'_i)$被

定义为

$$l(z_i, z_i') = -\log \frac{\exp(simi(z_i, z_i')/t)}{\sum_{k=1,k\neq i}^{b} \exp(simi(z_i, z_k)/t) + \sum_{k=1}^{b} \exp(simi(z_i, z'_k)/t)} \tag{3.1}$$

在(3.1)式中，*simi* 是表格相似度函数，*t* 是温度超参数。通过最小化同一个表格的原向量和增强后向量的之间的相似度得分，最大化不同表格之间的向量的相似度得分，即可得到较小的 *loss*。

在一个 batch 中，整体的损失函数总和$L_{contrast}$如(3.2)式所示

$$L_{contrast} = \frac{1}{2b} \sum_{k=1}^{b} [l(z_k, z_k') + l(z_k', z_k)] \tag{3.2}$$

3.3.2.4 Transformer 列语义相关模型

本文提出的 Transformer 列语义相关模型的目的是列无关向量数组转化为列相关向量数组。在实现过程中，本文使用对比学习专门设计并训练了一个包含六个隐藏层 Transformer 模型，用于执行列间语义建模任务。完整的模型结构如图 3-5 所示。

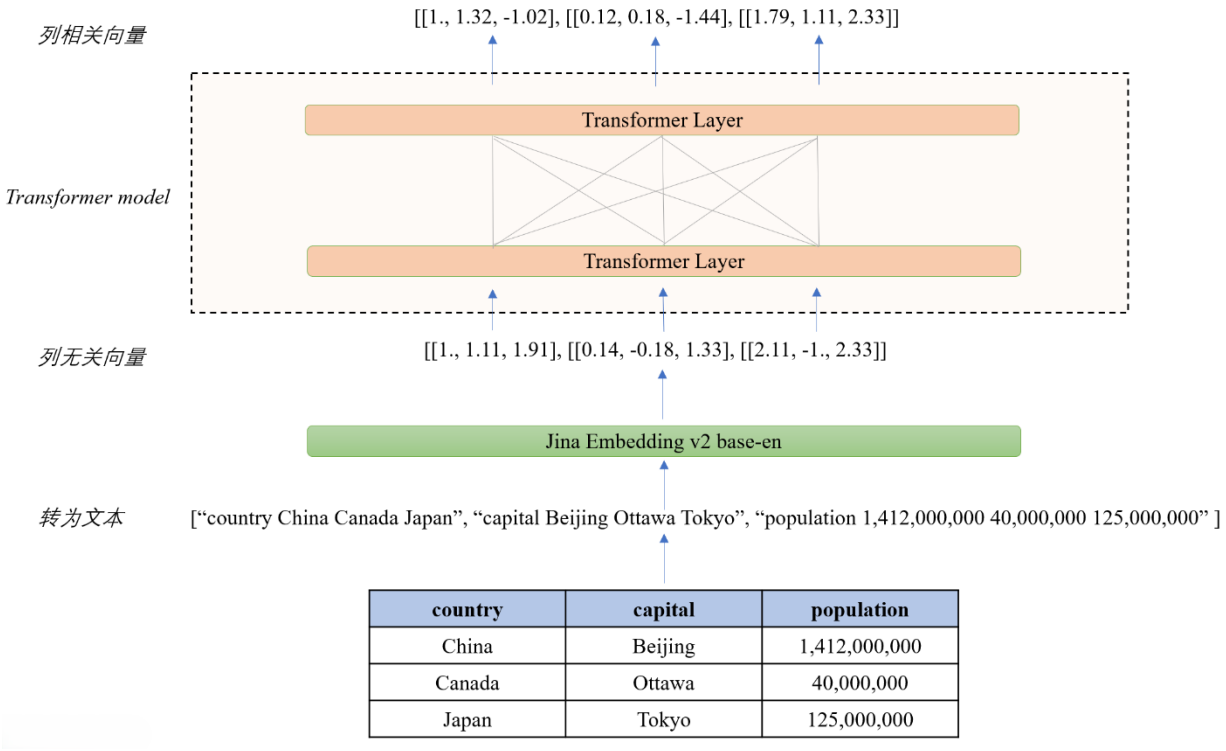


图 3-5 模型架构设计图

该 Transformer 模型的输入为一个维度为$n \times 768$的二维向量数组，其中 n 为表格的列数。输入向量经过模型的编码器和解码器后，输出同样维度为$n \times 768$的二维向量数组。通

过 Transformer 的自注意力机制获取一张表格内的列相关向量有效捕捉到列与列之间的语义关系，生成列语义相关向量，提升相关表格发现的准确性。

3.3.2.5 表格相似度

在本模块与相关表格模块中，识别和确定相关表格的关键在于计算表格之间的相似度。为了综合考虑表格各列之间的语义关系，从而得到整体的相似度评分，本文结合贪心算法，将表格间的相似度计算建模为一个加权二分图匹配问题。更具体而言，对于任意两张表格 S 和 T，本文构建一个二部图$G = \langle S, T, E \rangle$，其中 S 和 T 分别表示两张表格中的所有列，边集 E 则表示表格中任意两列之间的相似度得分。为消除不相似列对整体相似度得分的干扰，对于任意 $s \in S$ 和 $t \in T$，其边权 $E(s, t)$ 的计算方式如式 (3.3) 所示。

$$E(s,t) = \begin{cases} cosine_similar(T_{i,a}, T_{j,b}) & if\ cosine_similar(T_{i,a}, T_{j,b}) \geq \beta \\ 不存在 & else \end{cases} \tag{3.3}$$

得到二部图后，为了排除列数较多的表格的优势，两张表格 S 和 T 的相似度得分如公式(3.4)所示。在除以两张表格列数较多的数量后，能够消除列数较多的表格优势。

$$Simi(S,T) = \frac{max_{macth} G(S,T)}{\max(|S|, |T|)} \tag{3.4}$$

图 3-6 展示了一个计算表格之间的相似度的例子。在这个例子中，表 S 和表 T 的相似度得分过滤阈值被设置为 0.4，由于 *S3* 和 *T3* 的列向量相似度小于 0.4，因此这条边被舍弃。而在该例子中，被选择进行匹配的列之间被红色直线标出，所以这两张表格的相似度得分为$\frac{0.8+0.7+0.65}{4} = 0.5375$。

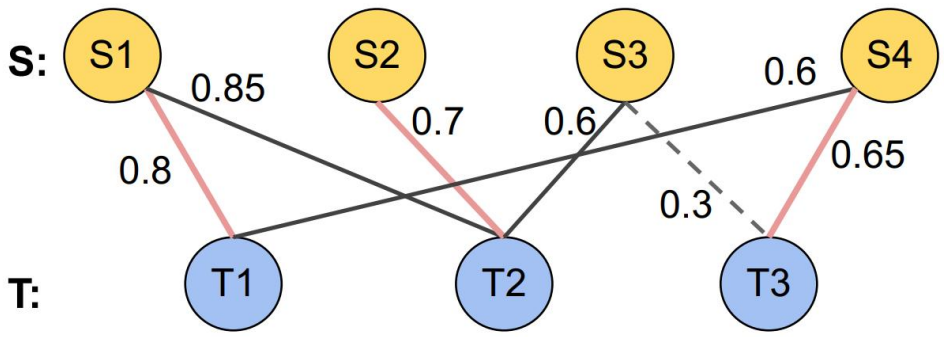


图 3-6 表格相似度得分计算示例

对于两张表格来说，直接计算它们的加权二部图匹配的时间复杂度非常高，时间复杂度为$O(n^4)$（使用 Dinic 算法转换为网络流计算），为了简化计算，本文使用了 Sinkhorn-

Knopp 算法简化计算，将一次计算一对表格的相似度得分的时间复杂度降低至了$O(n^3)$。

Sinkhorn-Knopp 算法是一种将非负矩阵通过交替的行列归一化转化为双随机矩阵的迭代方法，算法的核心思想是通过多次迭代的行列归一化操作，逐步逼近双随机约束下的最优解。双随机矩阵的特点是每行和每列的和均为 1，因此它在二分图最大匹配任务中具有重要意义。通过将原始相似度矩阵投转化为双随机矩阵，能够有效平衡行和列的权重分布，使得可以通过近似计算的方法获得二分图的最大匹配得分[22]。计算过程见算法 4 所示。

算法 *4*：获取两张表格的相似度得分

Input: 表格 S, 表格 T, 相似度阈值 β, Sinkhorn-Knopp 迭代次数 num_iter
Output: 表格相似度得分 score

```
for i from 1 to |S| do
    for j from 1 to |T| do
        similarity_matrix[i][j] = cosine_similarity(table1[i], table2[j])
similarity_matrix[similarity_matrix < β] = 0
for k from 1 to num_iter do
    for i from 1 to |S| do
        for j from 1 to |T| do
            similarity_matrix[i][j]/=sum(similarity_matrix[i][*])
            similarity_matrix[i][j]/=sum(similarity_matrix[*][j])
score=0
for k from 1 to min(|S|,|T|) do
    i, j = max(similarity_matrix)
    score+=similarity_matrix[i][j]
    similarity_matrix[i][*]=0
    similarity_matrix[*][j]=0
score/=max(|S|, |T|)
return score
```

3.3.2.6 HNSW 图构建

HNSW(Hierarchical Navigable Small World)是一种图结构算法，用于执行近似最近邻搜索。该算法基于多层级的图结构设计，通过构建多层稀疏图来实现高效搜索。其中，高层图提供粗粒度的导航功能，底层图则负责细粒度的精确搜索，从粗粒度到细粒度逐渐接近最优解。HNSW 图能够有效降低搜索过程中的遍历复杂度，同时保持较高的搜索效率和准确性。

传统的 HNSW 算法主要用于向量间的近似最近邻搜索，但是在本系统中表格相似度计算使得用的是自定义函数，为适应这种特殊相似度计算方式，本文改进了多层图结构。本文中，NSW 结构基于最小生成树原理，具体实现分为三步：首先按概率将数据点分配至不同层级；其次在每层内构建最小生成树，并添加长短边增强连通性；最后连接不同层中

编号相同的节点，形成跨层导航路径。这种设计既保留了 HNSW 的快速、精确搜索优势，又满足了自定义相似度计算的需求。具体实现见算法 5 所示

算法 5：构建 *HNSW-MST* 搜索图

Input: 表格间相似度矩阵 similarity_matrix, 表格数量 n, 图的层数 m, 超参数 κ
Output: HNSW-MST 搜索图 graph

```
graph = []
layer = []
for i from 1 to n do
    for j from 1 to m do
        if rand(0,1)<=κ^(m−j) do
            layer[i] = j
            break
for j from 1 to m do
    current_points = {i∈[1,n] where layer[i]=j}
    g = Minimum Spanning Tree of current_points
    foreach e∈current_points do
        if e∉g and e is a short edge do
            add e to g
        else if e∉g and e is a long edge do
            add e to g
    add g to graph
return graph
```

3.3.3 相关表格发现

3.3.3.1 简述

相关表格发现模块需要在 HNSW 图上查找相关表格，利用表格间相似度进行相关表格发现操作。由于直接遍历所有表格的方法效率太低，为提升效率，本文采用了多层图结构的近邻搜索策略。具体实现分为两步：首先，系统在 HNSW 图上执行逐层近邻搜索，最终到达底层，短时间内获取到大部分可能成为相关表格的表格相似度，避免了耗时的线性遍历操作。然后，系统对所有候选表格进行相似度评分排序，将得分最高的 k 张表格作为最后的相关表格输出。通过这两个步骤，既保证了结果准确性，又显著提高了查询效率。

3.3.3.2 算法流程

在查询过程中，系统会首先从第 1 层随机选择一个起始节点，并计算该节点与查询表格之间的相似度距离。然后，系统在每一层采用类似模拟退火的策略，即在当前节点的邻居中选择距离最小的节点进行转移，以尽可能接近该层的局部最优解。在完成当前层的搜索后，从所有节点中选取与查询表格距离最小的节点作为下一层的起始点，逐层向下执行该过程直至最底层。最终，根据所有表格的相似度得分，选取与查询表格最相近的前 k 个

节点作为相关表格返回。算法的伪代码见算法 6 所示。

算法 *6*：发现相关表格

Input: 查询表格 q_table, 数据湖表格 tables, HNSW 图 g, 需要返回的相关表格数量 k
Output: 相关表格下标 index

```
distance = [INF]
start = random choose a node from layer 1
distance[start]=Simi(q_table,tables[start])
for i from 1 to g.m do
    steps = |g.layer[i]| / 5
    while steps>0 do
        for v∈neighbour[start] do
            distance[v]=Simi(q_table,tables[start])
        strat = v∈neighbour[start] and distance[v] is minimum
        steps-=1
return index of top-k minimum element in distance
```

3.3.4 表格增强

3.3.4.1 简述

表格增强模块包括表格增强执行与表格空缺值填补两个子模块，该模块会在系统运行过程中基于相关表格与用户增强需求对用户查询表格进行合理增强。

在表格增强执行阶段，系统首先将用户查询表格、相关表格及提取的增强参数一起输入给 LLM，获取增强建议。随后，根据 LLM 输出的增强建议，表格增强模块采用“先 JOIN 后 UNION”的增强逻辑，调用 Pandas 库函数对原始表格进行增强，生成增强后的表格。为应对 LLM 输出结果的不确定性，模块中加入了格式校验与异常处理机制，以保证增强过程的稳定性与增强结果的正确性。

在表格空缺值填补过程中，系统首先根据增强参数确定数值型字段的填补策略，随后逐列判断数据类型，若为数值型字段，则使用填补策略进行填补；若为文本型字段，则调用预训练的 BERT 模型，结合上下文语义进行补全。整个过程在提升数据完整性的同时，也尽可能保留了原始表格的结构和语义信息。

3.3.4.2 表格增强执行

在表格增强执行过程中，相较于传统的表格增强方法，本系统引入了 LLM 以实现更为智能化的增强决策。表格增强执行阶段分为三个主要步骤。首先，系统会提取用户查询表和相关表格的关键信息，包括表头和部分数据，这些信息与增强参数一起被输入到 LLM 中。接着，LLM 按照预定格式输出增强策略，其中包含 JOIN 和 UNION 操作建议。然后，

系统获取并解析 LLM 给出的增强建议内容。最后，系统根据增强建议的解析结果，按照预设的增强逻辑对用户查询表进行增强处理，完成基于语义的表格结构扩展与表格增强。

预定的增强格式建议两个字段的JSON对象，分别为*join_operations*和*union_operations*。*join_operations* 是一个列表，列表中的每一项为一个子列表，表示一次 JOIN 操作。子列表结构可以表示为["查询表列","相关表编号.相关表列","相关表保留列 1","相关表保留列 2",...]，对于每个子列表来说，第一个元素为用户查询表中参与 JOIN 的列，第二个元素为相关表中参与 JOIN 的列（包含表编号），其余元素为在 JOIN 后需要保留在最终增强表格中的相关表列。*union_operations* 是一个列表，其长度为偶数。每两个相邻的子列表表示一次 UNION 操作，具体而言，第一个子列表说明了用于 UNION 的各列来源，格式为"相关表编号.列名"，多个列连接可以用+表示合并列；第二个子列表为每列在最终增强表格中的命名，对应上述列的位置顺序。

以下是一个增强建议的示例：*{"join_operations": [["department", "1.department", "department", "established date"]], "union_operations": [["2.staff name", "2.department"], ["name", "department"]]}*。在 *join_operations* 中，列表*["department", "1.department", "department", "established date"]*表示一次基于“*department*”字段的表连接操作：系统建议将用户查询表中的 *department* 列与编号为 1 的相关表中的 *department* 列进行 JOIN 操作，并在JOIN结果中保留相关表的*department*和*established date*两个字段。在*union_operations*中，两项列表表示一次字段合并操作。第一项*["2.staff name", "2.department"]*表示将编号为 2 的相关表中的 *staff name* 和 *department* 两列作为参与合并的源列；第二项*["name", "department"]*指定了这些列在最终增强表格中的命名，分别命名为 *name* 和 *department*。

在系统获取到增强建议后，会执行增强操作。系统首先处理 JOIN 增强操作，逐条解析增强建议，提取查询列、相关表编号、JOIN 列和需保留的列。系统使用 Pandas 的 merge 函数将相关表与查询表连接，同时清理冗余列。接着系统处理 UNION 操作，将每两个操作作为一组进行解析，统一设置新列名。系统会从多个相关表中提取数据并拼接，最后将拼接结果合并到原查询表中，完成数据增强。JOIN 操作和 UNION 操作的伪代码见算法 7 和算法 8 所示。

算法 7：执行 *JOIN* 增强操作

```
Input: 查询表格 query_table, JOIN 操作 join_operations, 相关表格 related_tables
Output: 增强后表格 query_table
foreach join_op in join_operations do
    query_column = join_op[0]
    related_ref = join_op[1]
    (table_id, related_column) = ParseTableRef(related_ref)
    keep_columns = Union(join_op[2:], [related_column])
    related_table = LoadCSV(related_tables[table_id])
    related_table = FilterColumns(related_table, keep_columns)
    related_table = DropDuplicates(related_table, related_column)
    query_table = MergeTables(query_table, related_table, on_left = query_column, on_right =
    related_column, method = "outer")
    query_table = DropDuplacateColumns(query_table)
return query_table
```

算法 8：执行 *UNION* 增强操作

```
Input: 查询表格 query_table, UNION 操作 union_operations, 相关表格 related_tables
Output: 增强后表格 query_table
for i from 1 to |union_operations| step=2 do
    operation_list = union_operations[i]
    column_names = union_operations[i + 1]
    involved_tables = ExtractTableIDs(operation_list)
    table_dfs = InitializeEmptyDictionary()
    foreach table_id∈involved_tables do
        table_dfs[table_id] = LoadCSV(related_tables[table_id])
    for j from 1 to |operation_list| do
        op = operation_list[j]
        new_col = column_names[j]
        if '+'∈op do
            ref1, ref2 = SplitJoinColumns(op)
            ApplyJoinBetween(table_dfs, ref1, ref2, new_col)
        else do
            tid, col = ParseTableRef(op)
            RenameColumn(table_dfs[tid], col, new_col)
    union_table = JoinUnionTables(table_dfs, operation_list)
    query_table = ConcatenateRows(query_table, union_table)
return query_table
```

3.3.4.3 表格空缺值填补

在完成表格增强后，由于存在增强过程中字段对齐不一致、数据缺失等问题，增强后的表格中往往会出现部分表格单元格为空的情况。为了提升表格数据的完整性与可用性，本系统设计并实现了空缺值填补模块对增强后表格中的缺失值进行处理。系统会根据用户指定的填补策略，自动识别并填补空缺字段，确保得到的增强表格在后续的下游机器学习

任务中具备更好的性能。

空值填补过程主要分为两步，首先，系统会先尝试将每一列转换为数值类型，便于后续统一处理。接下来，对非数值类型的列，系统会将缺失值替换为[MASK]，使用预训练 BERT 模型进行填充；而对于数值类型的列，则根据增强参数中的策略进行填补，策略包括使用随机森林回归模型进行预测、使用众数或中位数进行填充。空值填补过程的伪代码见算法 9 所示。

算法 9：空缺值填补

```
Input: 待填补表格 df, 填充策略 strategy
Output: 填补后表格 df
foreach column ∈ df do
    convert column to numeric type if possible
foreach column ∈ df do
    if column is not numeric do
        foreach value∈column and value=NA do
            filled_value = BERT_Predict(value)
            value = filled_value
foreach column ∈ df do
    if column is numeric do
        if strategy == "MODEL" do
            use RandomForestRegressor to predict missing values
        else if strategy == "MEAN" do
            fill missing values with mode
        else do
            fill missing values with median
return df
```

在空缺值填补过程中，系统会先填充文本列，再填充数值列。这样设计的的原因是文本列通常会影响数值列的含义，在结构化表格数据中，数值字段的值往往依赖于文本字段，因此先填补文本列可以使对于数值列填补时有更完整的上下文信息，提高数值预测的准确性。

3.3.5 数据存储

3.3.5.1 简述

数据存储模块在整个系统中承担核心的数据存储与管理功能。数据存储模块负责存储与管理的数据类型包括两种类型：数据湖表格的信息以及用户相关的全部数据。在本节内容中，将结合前文所述的各模块，对数据的存储方式进行分类分系统的详细阐述。

系统的存储方式主要分为两类：一类是采用 MySQL 数据库进行管理的结构化数据，另一类则是依托文件系统存储的非结构化或大小较大的文件数据。前者主要用于存储单条

数据大小较小、整体数据条数较多且对增删改查操作具有较高实时性要求的数据类型，如用户信息、数据湖表格信息以及用户表格增强的历史记录等。这类数据通常具备高并发访问的需求，适合通过关系型数据库进行高效管理。后者则适用于存储单条数据体大小较大、对操作速度要求较低的数据类型，如数据湖表格的向量信息以及用户头像图像等内容，文件系统更能胜任其容量与访问模式的需求。

3.3.5.2 MySQL 存储数据

在本系统中，MySQL 数据库有三张表，分别是用户信息表格，数据表格索引信息表格和用户增强历史记录信息表格。数据库表示例见图 3-7 所示。

#	username	password	avatar_path	user_type
1	[redacted]	[redacted]	[redacted]	user
2	admin	[redacted]	(NULL)	admin

(a) 用户信息表

#	table_id	table_path	pure_embedding_path	processed_embedding_path
1	1	E:/DLES_System/da...	E:/DLES_System/da...	E:/DLES_System/datalake_e..
2	2	E:/DLES_System/da...	E:/DLES_System/da...	E:/DLES_System/datalake_e..
3	3	E:/DLES_System/da...	E:/DLES_System/da...	E:/DLES_System/datalake_e..
4	4	E:/DLES_System/da...	E:/DLES_System/da...	E:/DLES_System/datalake_e..
5	5	E:/DLES_System/da...	E:/DLES_System/da...	E:/DLES_System/datalake_e..
6	6	E:/DLES_System/da...	E:/DLES_System/da...	E:/DLES_System/datalake_e..
7	7	E:/DLES_System/da...	E:/DLES_System/da...	E:/DLES_System/datalake_e..
8	8	E:/DLES_System/da...	E:/DLES_System/da...	E:/DLES_System/datalake_e..

(b) 数据表格索引信息表

#	username	history_tree
1	[redacted]	

(c) 用户增强历史记录信息表

图 3-7 数据库表示例

用户数据表格主要用于存储用户的基础信息，包括账户名、密码以及头像的 URL 地址等，主要在系统的登录与注册模块中被调用，并在其他模块中负责访问控制与用户鉴权。

数据湖表格索引信息在数据预处理模块、相关表格发现模块以及表格增强模块中均被使用。在数据预处理阶段，系统需将表格的自增 ID、数据湖表格的存储路径以及表格向量路径写入数据湖表格信息表中。而在相关表格发现与表格增强阶段，则通过查询该表快速获取数据湖表格的路径或表格向量路径，进一步实现基于文件系统的高效数据定位与访问。

用户增强历史记录信息表包含用户 ID 和用户的增强历史记录信息。历史信息字段采

用 JSON 类型字段以模拟文件系统中的层级存储结构。当用户对历史记录进行修改时，该字段也将与文件系统中的对应文件实现同步更新。

3.3.5.3 文件系统存储数据

文件系统负责存储数据湖表格、数据湖表格向量以及用户表格增强记录等相关信息其中，数据湖表格及其向量文件统一存储于统一命名文件夹中；用户表格增强记录则按照用户名，存储于“固定名称文件夹/用户名”路径下的目录中。

数据湖表格在系统运行前需准备完毕，向量的生成流程包含两步：首先，利用 Jina Embedding 模型生成初始表格向量，并暂存于临时目录下；然后，预训练列相关 Transformer 模型将列无关向量转换为列相关向量，并将其写入文件夹中供后续其他模块使用。

在数据预处理模块中构建 HNSW 索引图的过程中，系统需从数据库中获取向量路径，并据此从文件系统中加载对应的向量数据，以计算表格间的相似度并完成索引图的构建。而在表格增强模块中，则需根据相关表格模块提供的表格编号，根据索引读取对应的数据湖表格以执行最终的增强操作。

用户增强历史记录相关文件也用文件系统进行存储，每个用例以一个独立文件夹形式表示，内部包含四个文件：当前用例所使用的原始表格、其对应的初始向量、列相关向量，以及用户与 LLM 之间的对话历史记录。一个典型增强用例的目录结构示例如图 3-8 所示。



图 3-8 一条表格增强记录示例

3.3.6 下游任务验证

3.3.6.1 简述

在系统中，为了便于用户比较表格增强前后的效果，系统中内置了一个下游任务验证模块。该模块集成了随机森林模型，能够对用户上传的表格数据质量进行评估。用户只需上传数据并指定待预测列，系统就会自动将数据划分为训练集和测试集，完成模型训练与预测。对于数值类型的预测列，通过公式$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$计算均方根误差来量化

预测值与真实值之间的偏差；对于文本类型的预测列，通过公式$\mathrm{F1-Score}=2\times\frac{Precision\times Recall}{Precision+Recall}$计算 F1-Score 来量化预测值与真实值之间的偏差。评估完成后，系统会将结果以柱状图形式直观展示结果，帮助用户理解表格增强带来的性能变化。鉴于该模块并非本文的核心内容，这里不再展开详细讨论。

3.4 本章小结

本章系统全面地介绍了基于语义的数据湖表格增强系统，包括系统需求分析、系统架构设计和后端架构等。对于系统的五个关键模块，本章从功能维度分析到性能维度进行了详细的分析，明确了五个关键模块的具体需求。

在此基础上，结合第二章所提出的技术框架，本章设计了由数据预处理、相关表格发现、表格增强与空缺值填充构成的四个系统主要模块，并通过 HNSW 图与 LLM 提供的增强建议实现表格增强过程的自动化。然后，本章进一步明确了系统所依赖的技术框架与运行环境，包括 Python+Vue3，以及 Jina Embedding、Qwen 大模型等关键技术组件的选型依据。同时，通过软件流程图详细描述了表格向量化、语义增强及缺失值填充等核心流程的实现逻辑。

在技术框架介绍的基础上，本章介绍了系统三大核心模块的实现逻辑。表格预处理模块使用预训练 Embedding 模型和微调后的 Transformer 模型，将原始表格转换为高质量的列相关向量数组。该模块基于表格相似度构建 HNSW 图，以支持高效的近似近邻检索。相关表格发现模块在 HNSW 图的各层采用模拟退火搜索策略。该策略在当前节点的邻域内迭代选择距离最小的节点并跳转，逐层逼近局部最优解，最终返回 top-k 相关表格。表格增强模块分为两个阶段。第一阶段是增强执行，该步骤依据 LLM 生成增强建议，按照“先 JOIN，后 UNION”的顺序增强表格。第二阶段是空缺值填补，该步骤根据用户需求和字段类型，自动为数值字段和文本字段选择合适的填补方法，完成整个表格增强流程。

第四章 系统测试与结果分析

本章将结合前述系统设计与实现，对于基于语义的数据湖表格增强系统(Semantically-Driven Table Augmentation System for Data Lakes, SDASDL)进行了全面地功能性与性能测试，以验证本文提出方法的有效性与理论成果的可行性。

4.1 测试目标与测试配置

4.1.1 测试目标

本节旨在系统评估基于语义的数据湖表格增强系统(SDASDL)验证其在功能完整性、查询效率、增强结果有效性和整体增强速度等方面的表现。主要进行了以下五个方面的测试：

(1) 功能性测试

覆盖登录注册、相关表格发现、表格增强、下游任务评估等核心功能模块，通过自动化脚本模拟用户操作验证系统功能正确性与一致性。

(2) 相关表格发现性能测试

在三个不同数据集上，将本文提出的相关表格方向方法与 BM25、TF-IDF 等基线方法进行对比测试，主要评估指标包括平均查找时间、准确率和召回率。

(3) 表格增强性能测试

该测试主要针对下游任务增强效果，选择 WWk small 数据集开展实验，通过回归预测任务的性能对比，验证了基于 LLM 的智能增强方法相比传统方法的效果优势。

(4) 响应性能测试

评估系统在即席交互场景下的响应速度，测试从用户发出开始增强指令到表格增强结束全链路耗时，采取多次测试求平均值，默认使用相关表格数量 k=10。

(5) 稳定性测试

测试在表格增强模块中使用不同 top-k 表格数量下，下游任务性能整体变化趋势，评估系统对于输入相关表格变化趋势并分析原因。

4.1.2 测试配置

本系统的测试配置包括多个关键组件。表格向量化采用了 Jina Embedding v2 base-en 模型，数据的存储与处理则依赖 MySQL 和 Pandas。为提升表格数据的质量与表达能力，系统集成了 Pandas、Qwen 2.5-max 和 DeepSeek-V3 作为表格增强组件。在处理空缺值方面，

采用 scikit-learn 与 Google BERT(bert-base-uncased)共同作为填充工具。系统中的对比学习模块选用了一个包含 6 层 Transformer 层、8 个注意力头的基础模型作为骨干网络。在训练过程中，设置了批次大小为 32，学习率为 5e-5，最大 Token 序列长度为 512。所有实验均在一台配备了 NVIDIA RTX 3050 显卡和 Intel i5-12500H CPU 的计算机上完成。由于本地算力资源有限，系统运行所需的 LLM 均通过远程调用 API 接口的方式获取，未进行本地部署。

在测试过程中，首先在后端环境中通过 uvicorn 启动 FastAPI 服务；随后，在前端环境中使用 npm run dev 命令以开发模式运行 Vue 应用，从而完成系统的整体部署与启动。

本文使用了三个数据集对系统进行了评估。第一个是 Santos small 数据集，该数据集构建自加拿大开放数据平台，选取了其中 10 个表格作为原始数据，经过处理后共生成约 1530 张结构化表格。第二个和第三个是 WWk 系列数据集，该数据集维基百科页面中的表格构建而成。首先，从维基百科页面筛选结构化程度较高的表格，要求表格至少包含 10 行数据和 4 个有效列，确保表格具备足够的信息量。接着，为每个查询表查找并标注至少 5 个高度相关的目标表作为相关表格，保证在实验过程种每一张查询表格都拥有足够的相关表格。

为了适应不同规模的实验需求，WWk 数据集划分为两个子集：WWk small 包含约 550 张表格，WWk large 包含约 3500 张表格。这样可以支持同时在小规模和大规模场景下评估性能与准确性。

4.2 系统测试与分析

4.2.1 功能性测试

通过自动化脚本模拟用户操作，覆盖所有功能模块，确认了系统各个模块正确运行。系统可以正确完成登录注册、相关表格发现、表格增强、下游任务评估等核心功能任务。测试结果表明 SDASDL 各模块均正常运行，输出结果符合预期，可以支持大规模数据湖表格增强任务。

4.2.1.1 登录注册模块

如图 4-1 所示，为实现用户身份识别于鉴权，SDASDL 系统要求用户在使用前完成登录或注册；若用户遗忘密码，系统也提供基于邮箱的密码找回方式，以确保系统的可用性，提升了用户体验。通过脚本模拟用户进行注册、登录和找回密码操作，验证了系统可以正

确处理用户请求并且返回正确的结果，在进行注册或者找回密码后能够正确地登录系统，验证了登录注册模块的正确性和可用性。



(a) 登录系统



(b) 注册账号



(c) 找回密码

图 4-1 系统登录模块界面

4.2.1.2 表格增强模块

表格增强模块帮助用户完成表格增强操作。系统给用户提供了两种选择查询表格的方式，包括上传新表格或选择历史增强记录。如图 4-2 所示，操作界面分为三个主要区域：左侧显示历史记录，中间显示用户选择的查询表格具体内容，右侧提供与 LLM 的对话窗口。

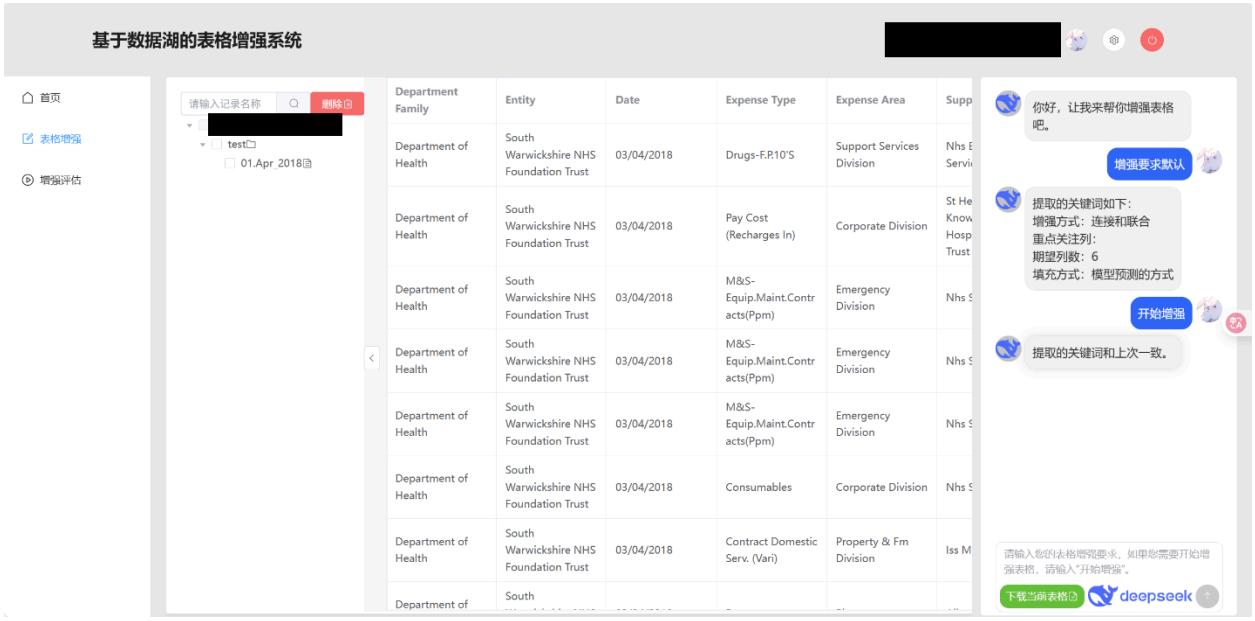


图 4-2 表格增强模块界面

用户通过对话窗口与模型交互，模型会自动分析需求并生成增强关键词。确认关键词无误后，用户输入“开始增强”指令即可执行后续增强操作。系统会首先发现相关表格，然后执行增强操作。在得到增强后表格后，系统会立即更新界面，在中央区域展示最新的表格结果。整个过程实现了从用户需求分提取到结果展示的自动化处理。根据表格增强流程图 4-3，编写脚本对于表格增强模块进行测试，验证了表格增强模块在处理不同大小的查询表格时候的稳定性和能够正确处理增强过程中的异常情况。

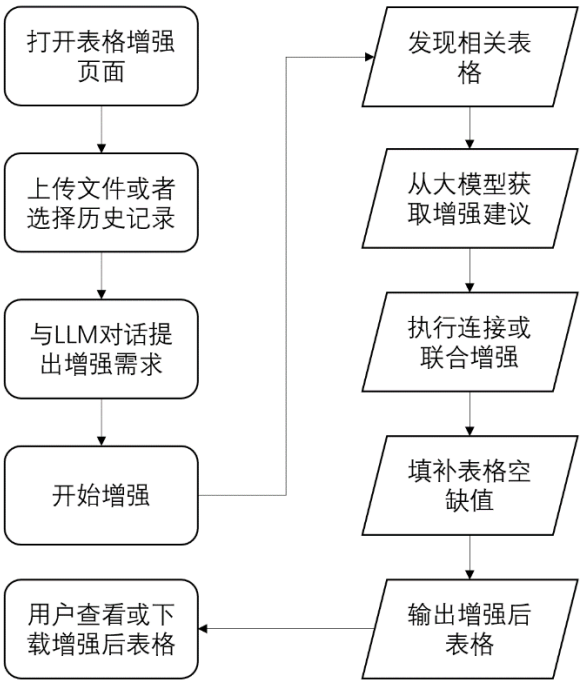


图 4-3 表格增强流程图

4.2.1.3 下游任务评估页面

下游任务评估模块包括两个核心功能：第一，系统在本模块内置了 XGBoost 模型用于进行预测任务评估，支持用户对增强效果进行量化分析；第二，系统在本模块集成了 LLM 代码编写功能，如果用户认为系统默认模型参数设置不足以满足评测需求，可通过与 LLM 交互获取定制化代码，实现更灵活的评估配置与实验对比。

多表格预测评估页面如图 4-4 所示。本页面支持用户在“表格上传”区域最多上传 5 个表格。所有已上传的表格会显示在“已上传文件”列表中，用户也可以在此列表中删除表格。在“训练参数”区域，用户可以输入模型训练的各项参数，同时用户需要选择一个表格公共列作为目标预测列。在用户点击“开始评估”按钮后，系统会根据预测列的数据类型自动选择评估指标。对于数值型数据，系统采用均方根误差作为评估指标；对于文本型数据，系统采用 F1-Score 作为评估指标。

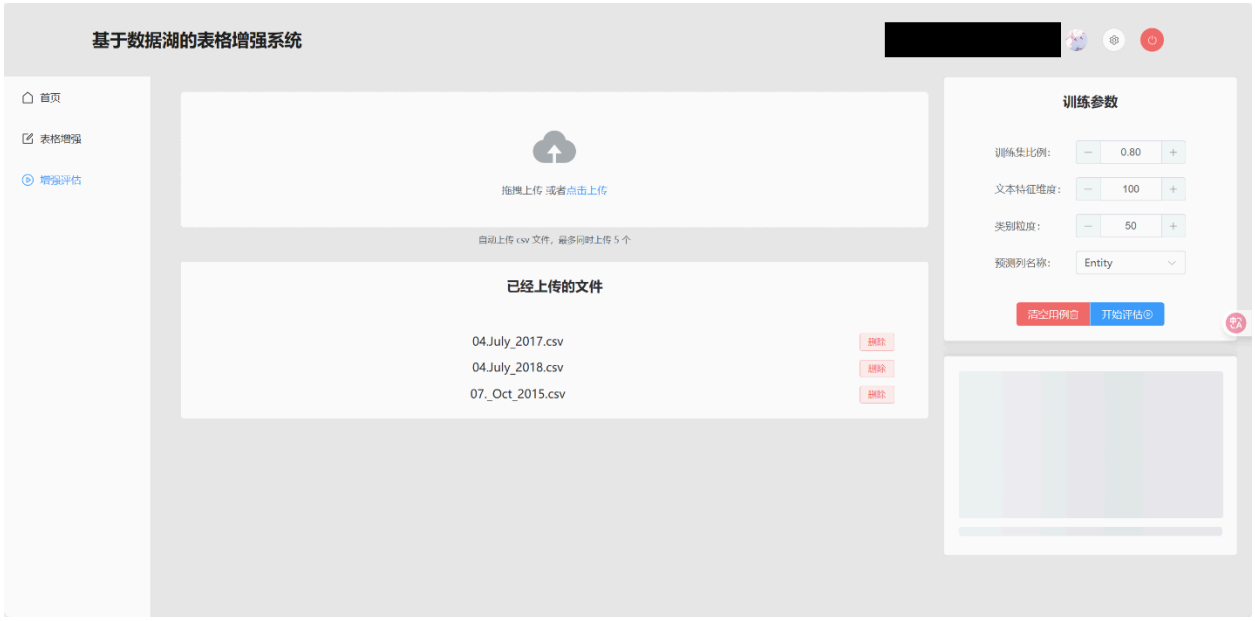


图 4-4 多表格预测评估页面

在评估完成后，系统会生成柱状图分别展示每一个表格的评估结果，如图 4-5 所示。这种可视化方式便于用户进行对比分析。



图 4-5 评估结果页面

根据下游任务评估流程图 4-6 编写自动化脚本，模拟用户添加、删除表格与调整评估参数等操作，验证了下游评估模块可以正确对于不同类型和大小的表格进行模型训练和结果预测，并且可以将预测的结果正确返回给用户，成功验证了本模块的正确性和对异常情况进行正确处理的能力。

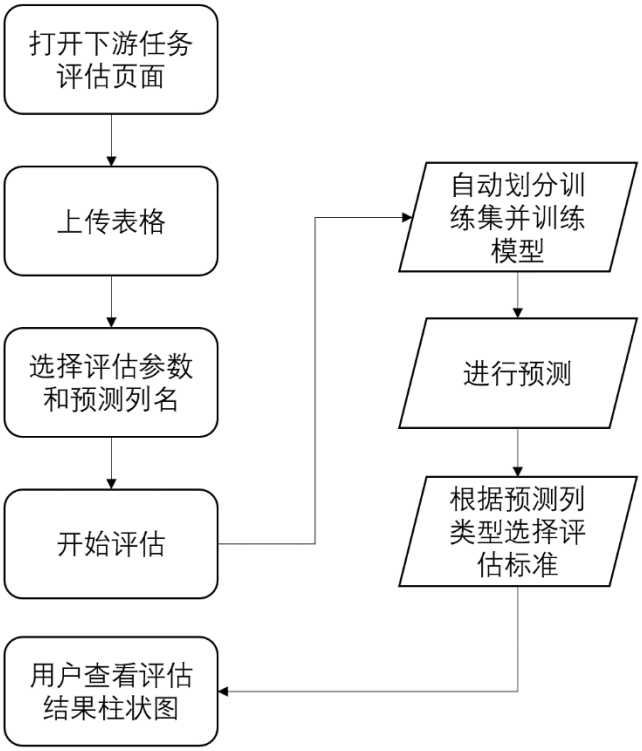


图 4-6 下游任务评估流程图

若用户有额外需求，系统也提供了“AI 帮写”功能以满足个性化需求，界面如图 4-7 所示。用户可在代码输入框中输入已有代码，并在需求输入区域输入具体的代码编写需求。在用户点击“上传”按钮后，系统将调用 LLM，根据用户需求与输入代码生成代码，最后将结果展示在代码输入框中，帮助用户快速完成代码编写。

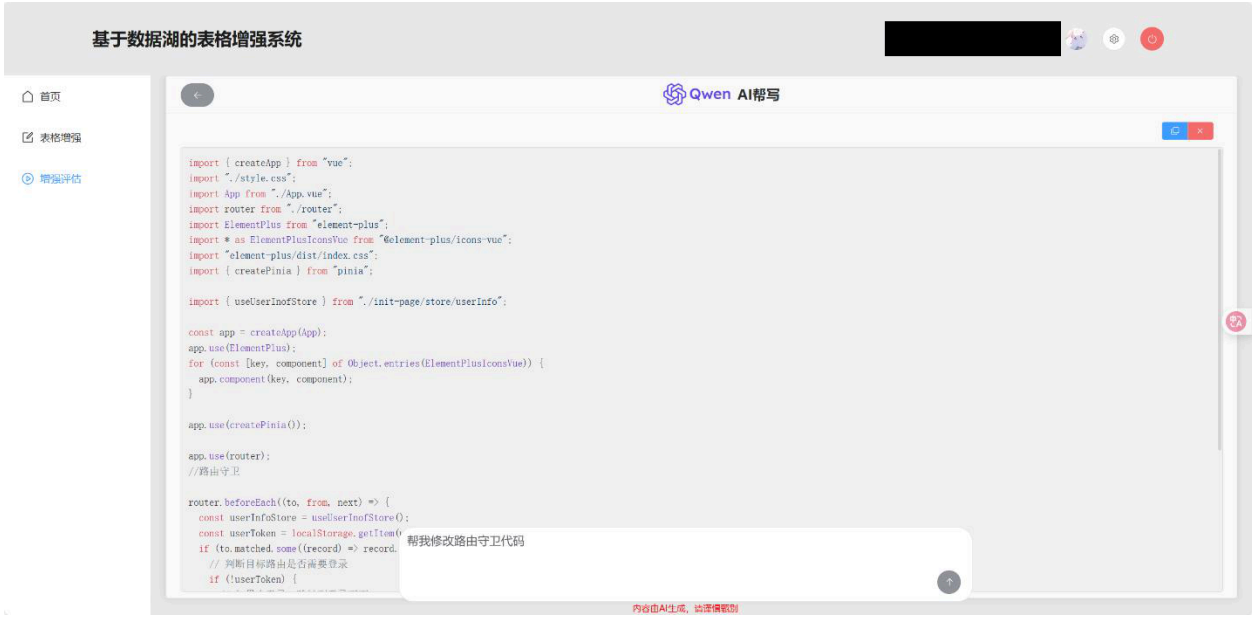


图 4-7 AI 帮写页面

4.2.2 相关表格发现性能测试

4.2.2.1 比较基准

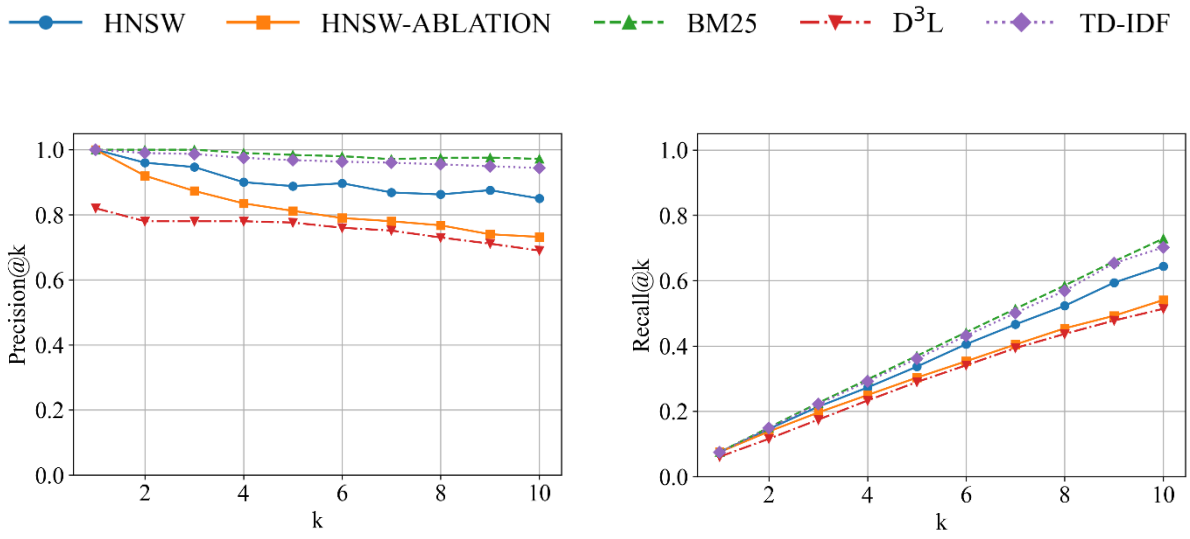
在相相关表格发现性能测试中，本论文提出的 HNSW-MST 方法与以下的基准进行了比较。

- TF-IDF：一种基于词在局部表格中频率和在全局文档中的出现次数的逆文档频率加权的特征表示方法，用于衡量表格间相似性。
- BM25：一种基于概率检索模型的打分函数，通过引入词频饱和效应与列长度归一化，优化了表格相似度的准确性。
- D^3L：一种表联合搜索方法，通过列名、值重叠和格式相似度等多个维度来查找相关表。

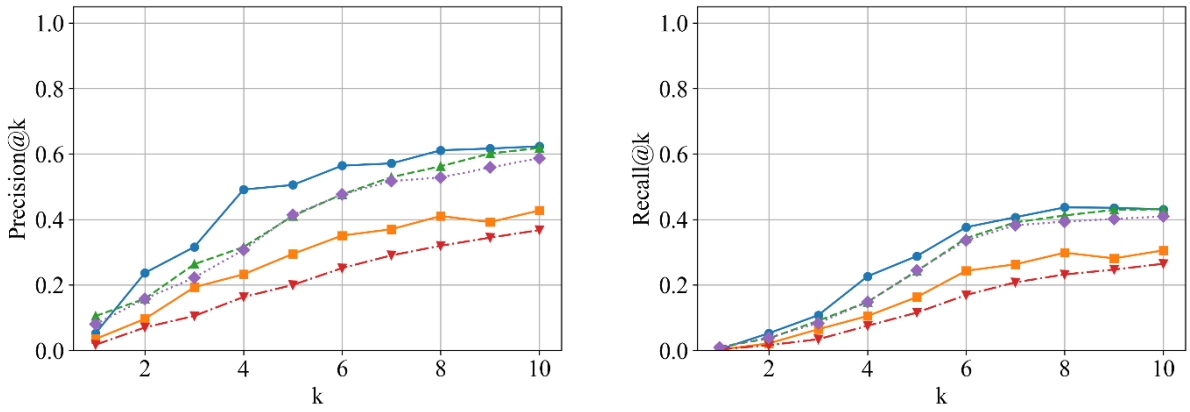
实验对比了本论文提出的方法与各基线方法。主要评估指标包括准确率和召回率，同时测试了不同数据集上的平均查找速度。通过这些对比，可以验证本论文提出的方法在准确性和效率上的优势。

4.2.2.2 测试结果

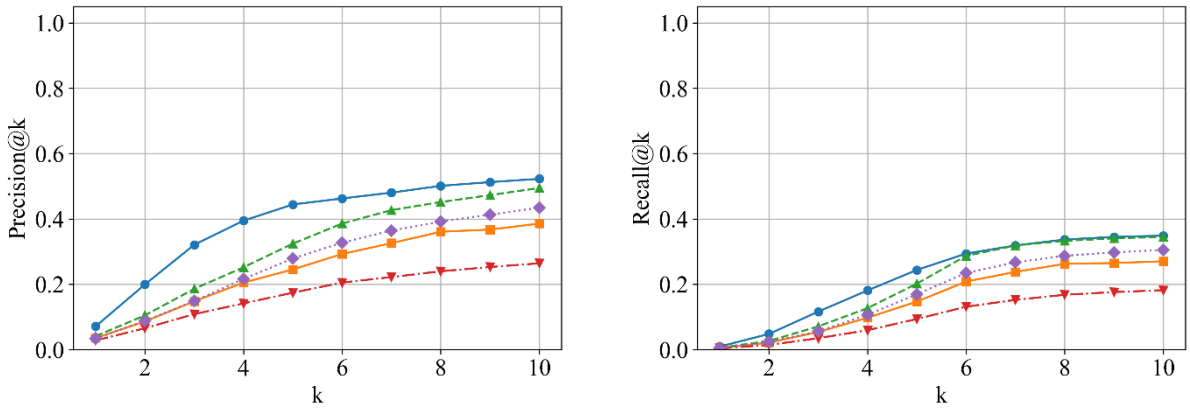
图 4-8 显示了在不同的三个数据集本系统使用的方法和基线方法在相关表格发现任务上的准确率和召回率的对比。



(a) P@*k* on Sanots small　　**(b) R@*k* on Santos small**



(c) P@*k* on WWk small **(d) R@*k* on WWk small**



(e) P@*k* on WWk large **(f) R@*k* on WWk large**

图 4-8 相关表格发现准确率与召回率对比

通过图 4-8 可知，在 WWk 数据集上，本文提出的基于 HNSW 的相关表格发现方法在准确率和召回率方面相较于基线中表现最优的 BM25 提升了约 10%。而在 Santos 数据集上，本文方法在表现上略逊于 BM25 和 TF-IDF 方法。这一差异可能源自于 Santos 数据集的构建过程较为简单。Santos 数据集是通过将大型表格随机拆分成小表格构建而成，导致相关表格之间存在较高比例的字段重复。因此，BM25 和 TF-IDF 等基于文本匹配的相关表格发现方法在这个数据集中具有一定优势。相反，在 WWk 数据集中，表格来源于维基百科页面，这些表格之间主要通过语义而文本匹配关联。由于本文提出的方法能够更有效地捕捉表格内及表格间的语义关系，因此在该数据集上相较于 BM25 和 TF-IDF 表现出更优的性能。

同时，本文通过消融实验验证了列语义关系建模的重要性。实验对比了完整 HNSW 方法与移除列语义建模 Transformer 模型的 HNSW-ABLATION 方法。结果显示，完整模型在

三个数据集上的表现均优于消融版本。具体而言，准确率和召回率平均提升幅度超过20%。这一差异充分证明，列语义建模能显著提升相关表格发现的准确性。

在查询延迟方面，本文提出的相关表格检索方法，在所有数据集上均显著优于基线方法，在各数据集中的平均查询延迟较其它方法缩短超过 50%，展示出极高的速度优势。各数据集上的具体查询延迟结果如表 4-1 所示。

表 4-1 平均查询延迟(s)

方法名称	Santos small	WWk small	WWk large
HNSW	**0.0213**	**0.0157**	**0.1492**
D^3L	0.7800	0.2959	0.6137
BM25	0.3055	0.0360	0.3301
TF-IDF	0.3515	0.0757	0.5782

表 4-1 显示，本文使用的方法在查询速度上优于所有基线方法。在 Santos 数据集上，本文使用的方法比最快的基线快 90%；在 WWk 数据集上，查询延迟降低超过 50%。以上数据证明了该方法在不同规模的数据集下都具有显著速度优势。通过对于不同数据集表格大小的分析表明，Santos 数据集表格平均大小较大，导致 BM25 和 TF-IDF 这类基于文本的方法查询延迟较高；而 WWk 数据集表格较小，使得 BM25 和 TF-IDF 方法的查询速度相对更快。

4.2.3 表格增强性能测试

4.2.3.1 比较基准

在表格增强性能测试中，本文提出的基于 LLM 的智能增强方法主要与以下几种表格增强方法进行比较。

(1) 简单基线连接：基于随机选择关联表的同名列进行直接连接，是最基础的连接策略。

(2) 统计基线连接：利用列的间相似度选择匹配性最强的列进行连接。

(3) 随机过滤连接：在排除唯一值过高或过低的列后，随机选择连接键进行左连接。

(4) 列名匹配基线连接：通过计算列名的字符串相似度来确定连接列。

在表格增强的对比实验中，本文提出的基于 LLM 的智能增强方法与各类基线方法在回归预测任务上进行了性能比较。具体评估指标包括平均平方误差(MSE)、均方根误差

(RMSE)、平均绝对误差(MAE)以及决定系数(R^2)。通过对这些误差指标的对比分析，表明了本文提出的方法在提升下游预测任务性能方面的有效性与优越性。

4.2.3.2 测试结果

表 4-2 展示了在 WWk small 数据集上，采用不同表格增强方法所生成的增强表格在下游回归预测任务中各项评估指标的平均提升幅度。

表 4-2 预测指标提升幅度(%)

方法	MSE	RMSE	MAE	R^2
基于 LLM 的智能增强	**46.15**	**46.15**	**46.15**	**65.38**
简单基线连接	19.23	19.23	23.07	15.38
统计基线连接	3.84	3.84	3.84	3.84
随机过滤连接	19.23	19.23	19.23	23.07
列名匹配基线连接	7.69	7.69	7.69	7.69

实验结果显示，基于 LLM 的智能增强方法在回归预测任务中由于其他基线方法。如表 4-2 所示，本文提出的方法在四个关键指标上取得显著提升，在 MSE、RMSE 和 MAE 上均提升了 46.15%，在 R^2 上提升了 65.38%，明显优于表现第二的简单基线连接方法。传统方法如统计基线连接和列名匹配基线的提升有限，表明仅依赖列名或唯一值比例的连接策略效果不佳。相比之下，本文提出的方法通过 LLM 深入理解表格间语义关系，大幅提升了预测任务表现，充分表明了其在表格增强方面的优势。

4.2.4 响应性能测试

为了测试 SDASDL 系统在实际使用场景的响应性能，本节测试了随着待增强的表格数量增加，系统整体及各模块耗时的变化情况，结果如图 4-9 所示。实验设置为，相关表格发现模块使用本文提出的类 HNSW 方法加速查找；表格增强模块每次增强时使用 top-10 相关表；测试表格数量由 50 张递增到 500 张。

测试结果表明，随着待增强表格数量的增强，SDASDL 系统的总耗时呈现线性增长趋势，从 15 秒增长至 155 秒，平均每张表格处理时间约 0.31 秒，且总耗时曲线与表格增强耗时曲线大致重合。这表明了系统的效率瓶颈主要在增强阶段，而相关表格发现耗时在百分之一秒级，可以忽略不计。在交互式实际使用场景中，用户增强一张表格耗时小于 10 秒，符合系统可用性要求。实验结果表明，SDASDL 系统在当前实现下已具备常规端到端延迟

和可用性。

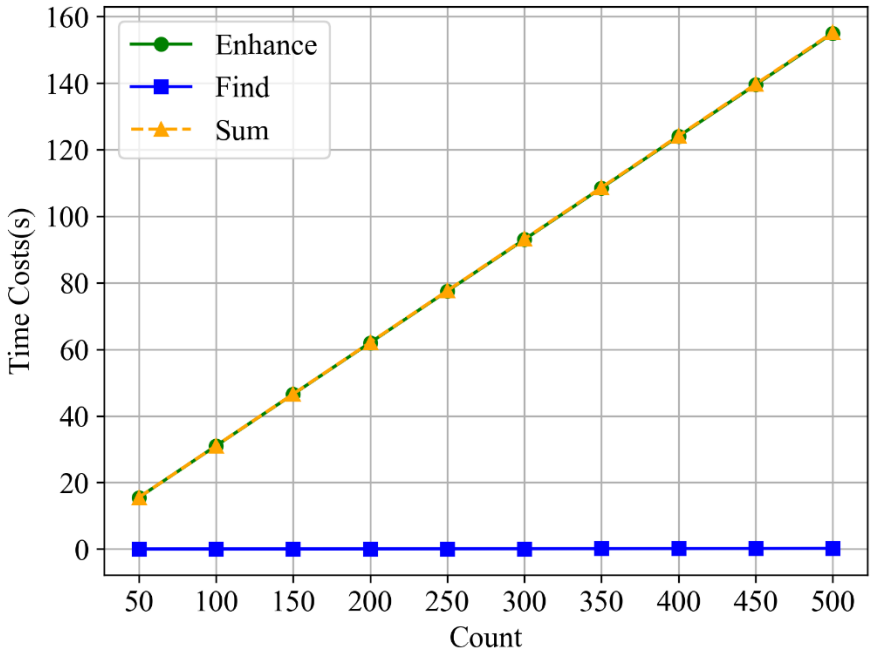
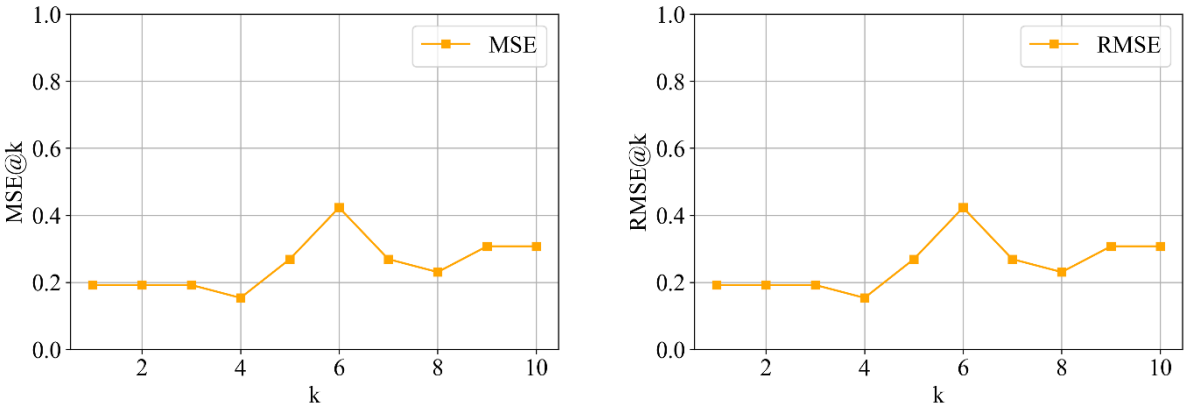


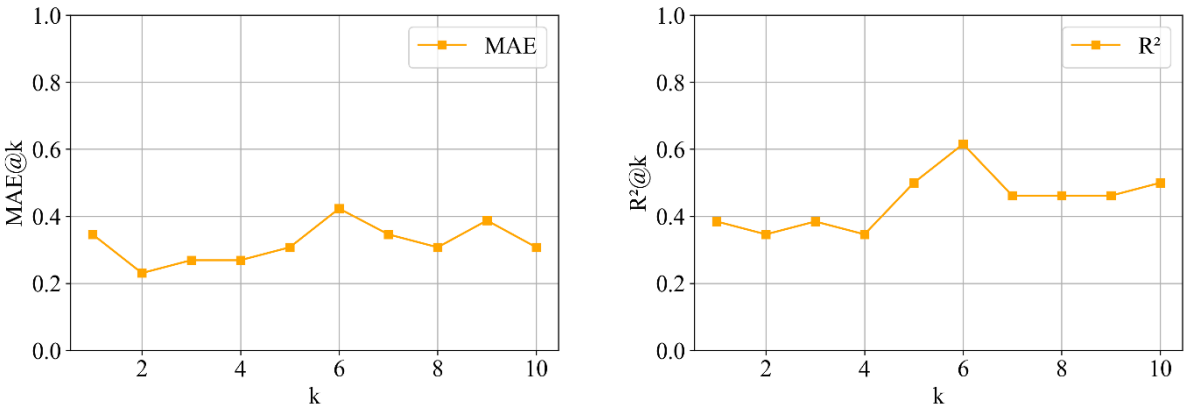
图 4-9 SDASDL 系统在不同表格数量下的耗时曲线

4.2.5 稳定性测试

遵循先前研究评估表格增强中相关表格数量 k 影响的惯例[1]-[14]，本文测试了候选表格数量从 1 递增到 10 时，下游机器学习任务的四项性能指标的变化情况，结果如图 4-10 所示。



(a) MSE **(b) RMSE**



(c) MAE **(d) R^2**

图 4-10 表格增强阶段使用不同数量相关表格增强性能

整体来看，增强性能曲线呈现“初期轻微震荡，然后上升，再下降稳定的趋势”，同时在 k=6 时，增强性能达到最高值。具体分析如下：当$k \leq 4$时，四项指标均较低且轻微震荡，说明此时可用相关表格信息不足，表格增强效果较差；当 k 提升至 6 时，整体系统性能达到最好效果，此时可用信息变多，LLM 在此信息量下提供最优增强建议。当 k 继续增大至 10 时，由于过多冗余信息的引入，LLM 的输入信息过多，过长的输入影响了 LLM 的决策能力，导致此时增强性能下降。

综合分析可知，SDASDL 系统的增强性能对 k 值存在明显的转折点特性：少量相关表格($k \leq 4$)提供信息较少，只能提供一些基础提升；当 k 继续升高时，效果变好，并且在 k=6 时达到峰值；当$k \geq 7$时，冗余信息过多，影响 LLM 进行决策，增强性能下降。因此在本系统实际使用中设置 k=6。

4.3 本章小结

本章通过多维度实验评估了本文提出的相关表格发现与表格增强方法。实验结果表明，该方法在语义较为复杂场景中具有显著优势：在 WWk 数据集上，相比最优基线 BM25，准确率和召回率提升约 10%，查询延迟降低 60%以上。消融实验显示，列级 Transformer 模型使相关表格发现准确率提升超过 20%，证明列间语义建模的重要性。在下游回归任务评估种中，基于 LLM 的智能增强策略表现优秀，四个指标均提升超过 40%，显著优于传统列名匹配方法。以上内容充分证明了本文提出的方法在准确性、效率和适用性方面的综合优势。

同时，本章系统性地介绍了基于语义的数据湖表格增强系统(SDASDL)的设计与实现，

表明其通过高效查询机制与 LLM 的合理使用，实现了自动化与智能化的表格增强。系统使用分层架构与前后端分离技术，保障了扩展性与运行效率。系统包含登陆注册模块、相关表格发现模块、表格增强模块和下游评估模块。然后，本章使用多种测试评估系统性能，验证了 SDASDL 系统在端到端表格增强流程中具有良好的功能完整性、可用性和准确性，证明了本文提出的方法的有效性和实用性。实验结果表明，SDASDL 系统能在保证系统稳定性的前提下，提升下游机器学习任务的性能，验证了本文提出的方法在实际数据湖环境中的可行性。

第五章 总结与展望

5.1 工作总结

在当前数据湖技术快速发展的背景下，结构化数据的规模持续扩大，结构化表格数量迅速增长。然而，如何高效利用数据湖中的结构化数据成为亟需解决的重要问题。传统的表格增强方法通常难以准确捕捉列与列之间的语义关系，且在大规模数据环境下存在较大的的时间开销。为此，本文提出并实现了一套基于语义的数据湖表格增强系统，以实现对数据湖中结构化表格的高效自动增强。本文的主要工作如下：

(1) 设计并实现了一套全流程自动化的基于语义的数据湖表格增强系统，涵盖了表格预处理、相关表格发现、增强策略生成与执行、空缺值填补等核心模块，系统整体具有较强的通用性与扩展性。

(2) 基于 TF-IDF 算法，提出了高效表格向量化方法，该方法能够将不同规模的表格统一转化为定长向量表示，有效压缩存储空间，同时在保留表格语义信息的前提下显著提升了向量化效率。

(3) 结合 HNSW 图索引结构，构建了基于自定义表格相似度的相关图，并设计了结合模拟退火算法的快速相关表格发现策略，支持对用户查询表的低延迟检索。

(4) 提出了基于 LLM 的强建议生成方法，设计了结构化的增强建议格式，实现了更智能化的表格增强流程，有效提升了增强结果的质量与可用性。

(5) 通过在多个真实数据集上的实验，系统性比较了多种相关表格发现方法和表格增强方法的性能表现。实验结果表明，本文所提出的方法在相关表格发现任务中具有更强的速度和准确性；在下游任务中，增强后的表格对于预测任务可带来超过40%的性能提升，验证了本系统的有效性与实用性。

5.2 工作展望

该系统已基本实现数据湖表格全流程增强的设计目标。在关键指标方面，系统达到了实用水平。这些指标包括相关表格发现的准确性、增强处理的速度以及查询延迟等。不过，系统目前仍存在一些不足。这些不足之处需要进一步探索和改进。

(1) 表格相似度定义仍有改进空间

当前系统的表格相似度定义方法能有效捕捉表格间的结构与语义关联。该方法在体现列间语义关系方面具有优势。但表格相似度是数值类型，难以直接使用高效的向量索引结

构进行快速查询。这在大规模表格集合中会影响相似表格的检索效率。未来研究可以探索将表格整体编码为单一向量的方法。这种方法需要在不损失语义表达力的前提下实现。这样既能保持表格表示的语义性，又能提高检索效率。

(2) 系统整体延迟仍有优化潜力

尽管当前系统对一张表格进行增强的时间已控制在 60 秒以内，基本满足功能需求，但由于受限于本地算力及 LLM API 的远程调用问题，系统整体的并发处理能力仍较为有限。为提升处理效率，未来可考虑在本地部署 LLM，结合更高性能的硬件资源以提高系统吞吐量，支持高并发的表格增强需求。

(3) 增强建议的格式有待进一步简化与标准化

当前的增强建议格式已具备结构性和可解释性，能够有效呈现增强操作的核心要素。然而，该格式在复杂场景下的表现仍有改进空间，其解析便捷性有待提升。另一个重要问题是，LLM 生成的建议可能出现列名不一致等错误，这些错误会影响增强任务的准确性。针对这些问题，后续工作可重点优化建议的表达形式。通过采用更结构化、更易解析的标准化数据格式，可以显著提升系统稳定性和自动化水平。

参考文献

[1] Khatiwada A, Fan G, Shraga R, et al. Santos: Relationship-Based Semantic Table Union Search[J]. Proceedings of the ACM on Management of Data, 2023, 1(1): 1-25.

[2] Bogatu A, Fernandes A A A, Paton N W, et al. Dataset Discovery in Data Lakes[C]. 2020 IEEE 36th International Conference on Data Engineering (ICDE). IEEE, 2020: 709-720.

[3] Nargesian F, Zhu E, Pu K Q, et al. Table Union Search on Open Data[J]. Proceedings of the VLDB Endowment, 2018, 11(7): 813-825.

[4] Chen P B, Zhang Y, Roth D. Is Table Retrieval a Solved Problem? Exploring Join-Aware Multi-Table Retrieval[C]. Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2024: 2687-2699.

[5] Zhu E, Deng D, Nargesian F, et al. Josie: Overlap Set Similarity Search for Finding Joinable Tables in Data Lakes[C]. Proceedings of the 2019 International Conference on Management of Data. 2019: 847-864.

[6] Harmouch H, Papenbrock T, Naumann F. Relational Header Discovery Using Similarity Search in a Table Corpus[C]. 2021 IEEE 37th International Conference on Data Engineering (ICDE). IEEE, 2021: 444-455.

[7] Santos A, Bessa A, Musco C, et al. A Sketch-Based Index for Correlated Dataset Search[C]. 2022 IEEE 38th International Conference on Data Engineering (ICDE). IEEE, 2022: 2928-2941.

[8] Dong Y, Takeoka K, Xiao C, et al. Efficient Joinable Table Discovery in Data Lakes: A High-Dimensional Similarity-Based Approach[C]. 2021 IEEE 37th International Conference on Data Engineering (ICDE). IEEE, 2021: 456-467.

[9] Esmailoghli M, Quiané-Ruiz J A, Abedjan Z. MATE: Multi-Attribute Table Extraction[J]. Proceedings of the VLDB Endowment, 2022, 15(8): 1684-1696.

[10] Chepurko N, Marcus R, Zgraggen E, et al. ARDA: Automatic Relational Data Augmentation for Machine Learning[J]. Proceedings of the VLDB Endowment, 2020, 13(9).

[11] Zhao Z, Castro Fernandez R. Leva: Boosting Machine Learning Performance with Relational Embedding Data Augmentation[C]. Proceedings of the 2022 International Conference on Management of Data. 2022: 1504-1517.

[12] Castelo S, Rampin R, Santos A, et al. Auctus: A Dataset Search Engine for Data Discovery and Augmentation[J]. Proceedings of the VLDB Endowment, 2021, 14(12): 2791-2794.

[13] Khatiwada A, Shraga R, Miller R J. DIALITE: Discover, Align and Integrate Open Data Tables[C]. ACM SIGMOD. 2023, 1(1).

[14] Fan G, Wang J, Li Y, et al. Semantics-Aware Dataset Discovery from Data Lakes with Contextualized Column-Based Representation Learning[J]. Proceedings of the VLDB Endowment, 2023, 16(7).

[15] Stekhoven D J, Bühlmann P. MissForest—Non-Parametric Missing Value Imputation for Mixed-Type Data[J]. Bioinformatics, 2012, 28(1): 112-118.

[16] Yoon J, Jordon J, Schaar M. Gain: Missing Data Imputation Using Generative Adversarial Nets[C]//International Conference on Machine Learning. PMLR, 2018: 5689-5698.

[17] Zhu E, He Y, Chaudhuri S. Auto-Join: Joining Tables by Leveraging Transformations[J]. Proceedings of the VLDB Endowment, 2017, 10(10): 1034-1045.

[18] Devlin J, Chang M W, Lee K, et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding[C]. Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies. 2019: 4171–4186.

[19] Vaswani A, Shazeer N, Parmar N, et al. Attention is All You Need[C]. Advances in Neural Information Processing Systems. 2017, 30.

[20] Chen T, Kornblith S, Norouzi M, et al. A Simple Framework for Contrastive Learning of Visual Representations[C]. Proceedings of the 37th International Conference on Machine Learning. 2020: 1597–1607.

[21] Günther M, Ong J, Mohr I, et al. JINA EMBEDDINGS2: 8192-Token General-Purpose Text Embeddings for Long Documents[EB/OL]. arXiv preprint arXiv:2310.19923, 2023.

[22] Sinkhorn R. A Relationship between Arbitrary Positive Matrices and Doubly Stochastic Matrices[J]. The Annals of Mathematical Statistics, 1964, 35(2): 876–879.